



IO MANAGER

APP MULTIMODAL PARA IO INSTRUMENTS

AGUSTÍN C. FERNÁNDEZ VÁZQUEZ
DAW/ZV2 - PROYECTO INTERMODULAR
IES ZAIDÍN VERGELES - GRANADA - 2026

ÍNDICE

1. INTRODUCCIÓN

1.1 Objetivos Técnicos

1.2 Sobre hombros de gigantes

1.3 IO-8 Synth

2. IO MANAGER

2.1 Tecnologías y entorno de desarrollo

2.2 Recorrido por las funcionalidades

2.2.1 Landing Page

2.2.2 Profile

2.2.3 Nav, Status bar y Presets bar

2.2.4 IO Control

2.2.5 IO Cloud

2.2.6 IO Firmware

2.2.7 About

2.3 Desarrollo de la App

2.3.1 Arquitectura

2.3.2 Base de datos

2.3.3 Auth y roles

2.3.4 Seguridad

2.4 Diseño y UX

2.4.1 La repercusión actual del Retrofuturismo

2.4.2 Logo y línea de diseño

2.4.3 Adaptabilidad

2.4.4 Accesibilidad

2.4 Integración ESP32

3. CONCLUSIONES

3.1 Análisis de resultados

3.2 To do

4. DOCUMENTACIÓN, ANEXOS Y WEBGRAFÍA

1. INTRODUCCIÓN

1.1 Objetivos técnicos

El presente proyecto consiste en el desarrollo de una aplicación host integral diseñada para la gestión y control de un sintetizador de audio embedded basado en el microcontrolador ESP32. La arquitectura propuesta desplaza la complejidad de la interfaz de usuario y la persistencia de datos masiva hacia un entorno de escritorio, permitiendo que el hardware se dedique exclusivamente a la síntesis de señal y la gestión de periféricos.

El sistema se fundamenta en tres pilares operativos:

1. Gestión avanzada de archivos: Implementación de un sistema de librerías de sonido (presets) almacenados de forma redundante en bases de datos relacionales y sistemas de archivos locales (LittleFS).

2. Control MIDI en tiempo real: Flujo de mensajes de baja latencia para la manipulación de parámetros de síntesis mediante protocolos de comunicación inalámbrica.

3. Mantenimiento mediante actualizaciones: Capacidad de actualizar el firmware y los recursos estáticos del dispositivo, garantizando la escalabilidad del sistema.

El objetivo primordial es orquestar una comunicación fluida entre tres entornos distintos: el microcontrolador, la lógica de datos y la UI. La aplicación debe asegurar que cualquier cambio realizado en la superficie de control se refleje instantáneamente en el hardware y se registre de forma coherente en la base de datos, manteniendo la integridad de los estados en todos los nodos del sistema.

En un contexto musical, la respuesta táctil es crítica. Uno de los objetivos principales es minimizar el jitter y el retardo en la transmisión de mensajes MIDI. Para ello, se establece un flujo de datos asíncrono que prioriza el tráfico de control sobre el tráfico de gestión, utilizando WebSockets para permitir una manipulación de parámetros sonora imperceptible al oído humano (latencia inferior a 10-15 ms). El sistema debe garantizar que los ajustes sonoros (presets) sobrevivan a ciclos de apagado o fallos de conexión. Lo que implica almacenamiento simultáneo en el host y en el dispositivo (LittleFS), con una conversión de datos de la interfaz a estructuras binarias optimizadas para reducir el desgaste de la memoria flash del microcontrolador.

Esta aproximación se inspira y se contextualiza dentro de un marco más amplio de desarrollo de software y hardware musical, tomando referencias de proyectos consolidados como Audacity, por su madurez en la edición y gestión de audio open-source; Dexed, por su portabilidad a microcontroladores y Raspberry Pi y su enfoque en síntesis FM; y los desarrollos de Marcel Licence, por su especialización en síntesis embebida, polifonía en microcontroladores y herramientas MIDI para hardware DIY.

1.1 Sobre hombros de gigantes

Las principales referencias e influencias para comenzar con este proyecto de síntesis digital sobre microcontroladores fueron las siguientes:

Marcel Licence:

Marcel Licence es un desarrollador open-source centrado en la síntesis de audio y el mundo DIY de microcontroladores. Es el creador de la biblioteca ML_SynthTools, que bajo licencia GPL-3.0 proporciona módulos para generar sonido, manejar codecs de audio, efectos, sintetizadores (“organ”, osciladores, arpegiador), MIDI y más.

Los proyectos de Marcel Licence son compatibles con múltiples plataformas embebidas: ESP32 (incluyendo variantes), ESP8266, RP2040 (Raspberry Pi Pico), Teensy, STM32, entre otros.

Entre sus proyectos destacados están un sintetizador básico para ESP32 con polifonía y entrada MIDI, adaptador MIDI USB para ESP32, y ejemplos de sintetizador tipo órgano y piano electrónico para microcontroladores.

Además, su trabajo ha sido reconocido en la comunidad DIY: por ejemplo, en Arduino Audio and MIDI Frameworks se destaca su biblioteca como una de las más completas para síntesis en microcontroladores.

Marcel Licence también ha creado secuenciadores MIDI ligeros para ESP32, lo que demuestra su interés no solo en la generación de sonido sino también en la interacción musical en tiempo real. Representa un perfil muy técnico y especializado en síntesis embebida, con un fuerte compromiso con el código libre, la portabilidad y la comunidad maker.

Dexed:

Dexed es un sintetizador FM open-source inspirado muy de cerca en el Yamaha DX7. Su repositorio principal está en GitHub: combina el motor MSFA (Music Synthesizer For Android) con una interfaz plugin, usando JUCE. Está bajo licencia GPL-3.0, lo que facilita que la comunidad lo adapte y portee a otros entornos.

Según Future Music España, el desarrollador central es Pascal Gauthier, también conocido como “Digital Suburban”. Además, Dexed permite cargar y exportar programas SysEx de los DX7 originales, manteniendo compatibilidad con el hardware Yamaha real, lo cual es muy valioso para los puristas.

Audacity:

Audacity es un editor de audio libre y multiplataforma con casi 25 años de historia. Permite grabar, editar y mezclar pistas de audio digital de manera flexible y accesible, tanto para usuarios domésticos como para proyectos educativos.

Su licencia GPL-3.0 y su comunidad activa facilitan la colaboración y la extensión mediante plugins y efectos. Aunque carece de algunas funcionalidades profesionales avanzadas de DAWs completos y ha tenido polémicas recientes sobre telemetría y privacidad, sigue siendo un referente en software de audio libre, con amplia compatibilidad de formatos y soporte multiplataforma (Windows, macOS, Linux). Audacity es especialmente valorado en entornos educativos y por desarrolladores que buscan un proyecto open-source maduro como base o referencia.

1.2 IO-8 Synth

Ío es una luna de Júpiter y Jupiter 8 de Roland, uno de los sintetizadores más famosos de todos los tiempos. Hace aproximadamente 3 años, sin tener demasiada idea ni de síntesis ni de programación, me embarqué en un proyecto que cambiaría por completo mi sistema de intereses. Empecé a probar algunos sketches de sintetizadores digitales en Arduino, en concreto, los ejemplos básicos de los repositorios de Marcel Licence.

Como proyecto experimental y de aprendizaje, comencé a escribir algunas líneas hasta que todo un mundo de síntesis, generación de ondas, anti-aliasing, filtros y otras aplicaciones se abrió delante de mí. A la par, también lo hizo el mundo de la programación.



La idea está definida. Ocho voces de polifonía con tres osciladores por voz, en entorno Arduino pero con tweaks de freeRTOS para una ESP32 - WROOM, uno de los microcontroladores más básicos de la serie. Tiene dos núcleos a 240 MHz y aproximadamente unos 350kB de memoria dinámica útil. Estas limitaciones me han hecho llevar la placa al límite y aprovechar al máximo cada recurso, hasta conseguir que todo el motor de audio corra en un solo core de forma secuencial y determinista, ejecutando 48000 samples por segundo a 32 bits de resolución en menos de 1950 microsegundos. Sin tablas de ondas; reduciendo el aliasing al mínimo posible, generando en tiempo real ondas recortadas con una técnica que no existía hasta el momento. No ha sido fácil.

ESP32 Ha demostrado ser un microcontrolador polivalente, encasillado en aplicaciones IoT, pero con un potencial mucho mayor. No tiene 1 GB de RAM como una Raspberry y no tiene los 800MHz de CPU de una Teensy, pero es increíblemente fácil de programar, teniendo en cuenta que está perfectamente integrada dentro del entorno Arduino. Todo lo demás es creatividad y warnings del compilador por no hacer las cosas de la forma más ortodoxa.

Ahora queda el otro núcleo, que es el que se encarga de la pantalla TFT, el MIDI clock, el procesamiento de las señales MIDI externas, el despliegue del servidor web asíncrono para la comunicación por Websockets con la App y la gestión interna de los presets (configuraciones de sonido) mediante un sistema de archivos basado en la biblioteca LittleFS.

2. IO MANAGER

2.1 Tecnologías y entorno de desarrollo

El proyecto IO-Manager es una aplicación híbrida navegador / desktop. A lo largo de su ciclo de desarrollo, el proyecto ha evolucionado desde una arquitectura monolítica web hacia un modelo desacoplado e híbrido de escritorio de alto rendimiento. En este apartado, detallo las decisiones arquitectónicas, la ingeniería de la cadena de herramientas (tooling), la gestión de entornos de desarrollo y las soluciones aplicadas ante los desafíos técnicos globales y locales de seguridad y compatibilidad.

Evolución del Stack Tecnológico y Arquitectura:

El ecosistema de IO-Manager ha sufrido una transformación crítica impulsada por las restricciones de los entornos de ejecución nativos. Quedando así preparada para la integración en Tauri.

```
[Arq. Inicial]  Laravel Monolito → Inertia.js → React (Auth → Cookies de Sesión)
                ▼ (Bloqueo por directivas CORS / Restricción de Entorno WebView)
[Arq. Final]   Laravel RESTful API (Sanctum Tokens) ← Axios → React (Desktop) / Tauri
```

El Ecosistema del Backend: Laravel Sail y PHP 8.4:

Se seleccionó Laravel 13 sobre PHP 8.4 mediante un entorno contenedorizado en Docker con Sail. Esta solución garantiza la paridad de entornos de desarrollo y producción de forma inmediata. El esquema inicial integraba motor de base de datos, caché y colas Redis y Servidor de pruebas de correo (Mailpit).

Inicialmente, se optó por una pila monolítica moderna utilizando Laravel Breeze con React e Inertia.js. Sin embargo, la necesidad de compilar un binario de escritorio nativo generó un conflicto arquitectónico insalvable: Al empaquetar el frontend para escritorio, la aplicación se ejecuta fuera del dominio web tradicional (entorno file:// o esquemas personalizados de WebView). Esto rompe la persistencia de autenticación basada en cookies de sesión e introduce bloqueos estrictos de CORS (Cross-Origin Resource Sharing).

Por esta razón, se eliminó por completo la capa de Inertia.js y se migró hacia una arquitectura de API RESTful. La autenticación por sesión mutará a Bearer Tokens mediante Laravel Sanctum, donde el token emitido (plainTextToken) es capturado por el frontend y persistido mediante los mecanismos locales de almacenamiento de la app de escritorio. Las comunicaciones se delegaron por completo a la biblioteca Axios, previa publicación y flexibilización de las políticas en config/cors.php.

El Descarte de NativePHP en Favor de Tauri:

El plan original contemplaba el uso de NativePHP (Electron/Framework). No obstante, las restricciones de compatibilidad del paquete con las dependencias contractuales de las versiones más recientes de Laravel (illuminate/contracts) provocaron un fallo de resolución en el gestor de dependencias Composer.

Ante este bloqueo, se eligió Tauri (núcleo en Rust y frontend en React). Los argumentos técnicos para esta elección fueron la reducción drástica del tamaño del instalador final (~10 MB) y del consumo de memoria RAM al reutilizar el motor de renderizado nativo del sistema operativo (WebView en macOS) en lugar de embeber una instancia completa de Chromium (como hace Electron) y la compatibilidad multiplataforma. El nuevo stack ofrece una solución nativa del problema de compilación cruzada gracias al tipado estricto y el manejo de memoria seguro de Rust.

La resolución del conflicto de compilación entre los entornos aislados de Docker Linux (linux-x64 requeridos por Sail) y las necesidades de enlace del sistema operativo anfitrión Mac (darwin-x64 requeridos por Tauri) se solventó configurando a Vite para que se ejecute directamente desde la máquina local durante las fases de diseño y empaquetado.

Con el direccionamiento de salida corregido en el archivo de propiedades del motor de escritorio ("frontendDist": "../dist/"), la ejecución del comando `npx tauri build` genera de forma exitosa los paquetes ejecutables compilados de grado de producción con un consumo mínimo de recursos en disco y memoria, y con una infraestructura IoT escalable y segura.

Orquestación y Automatización (macOS Automator)

Para mitigar la fricción operativa de levantar múltiples servicios manualmente, se diseñó un sistema de automatización mediante la herramienta Automator de macOS, orquestando scripts de shell y AppleScript para controlar el ciclo de vida de la aplicación.

El script de inicio (Start project environment.app) inicializa de forma silenciosa Docker Desktop, valida su estado activo mediante un bucle de comprobación (`docker info`), levanta el entorno de contenedores en segundo plano (`sail up -d`), arranca el servidor de desarrollo de Vite (`npm run dev`) e iconiza automáticamente la ventana de la terminal y de Docker para mantener el espacio de trabajo limpio.

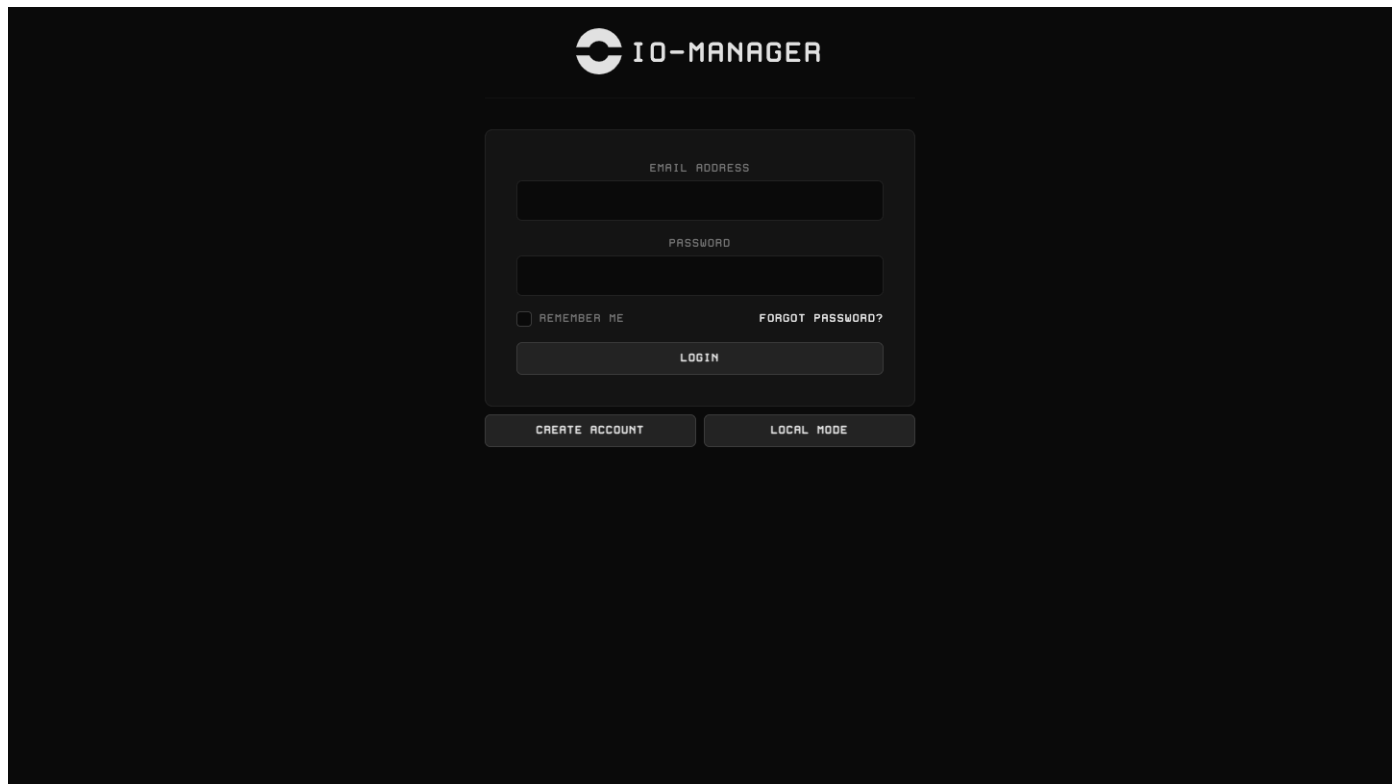
El script de cierre (Stop project environment.app) detiene los contenedores de manera controlada (`sail stop`), realiza un seguimiento en caliente de los procesos activos de los contenedores asociados a io-manager y, tras confirmar el apagado, finaliza de manera limpia la aplicación Docker Desktop y cierra los terminales residuales.

Mitigación de Vulnerabilidades NPM

El proyecto se enfrentó a una alerta de seguridad crítica global en marzo de 2026: el compromiso de múltiples paquetes de NPM (incluyendo versiones específicas de Axios y TanStack) inyectados con software de espionaje malicioso. La respuesta de ingeniería fue inmediata y contundente, estableciendo políticas estrictas de control de daños. Se eliminaron los prefijos de actualización automática del archivo package.json, fijando las dependencias exclusivamente a versiones seguras verificadas. Para ejecutar el saneamiento de entorno: Se ejecutó una purga absoluta del árbol de dependencias dentro del aislamiento del contenedor Docker mediante procedimientos de limpieza forzada.

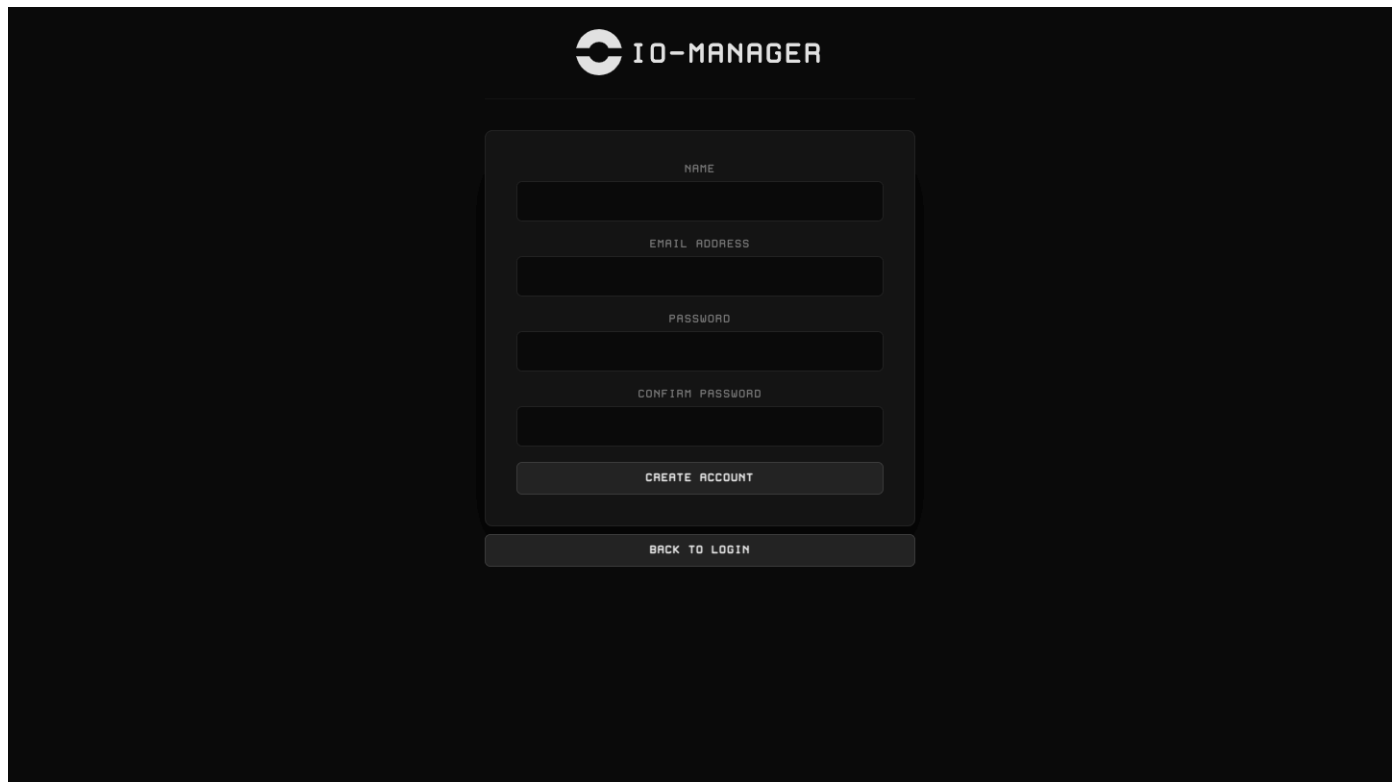
2.2 Recorrido por las funcionalidades

2.2.1 Landing Page



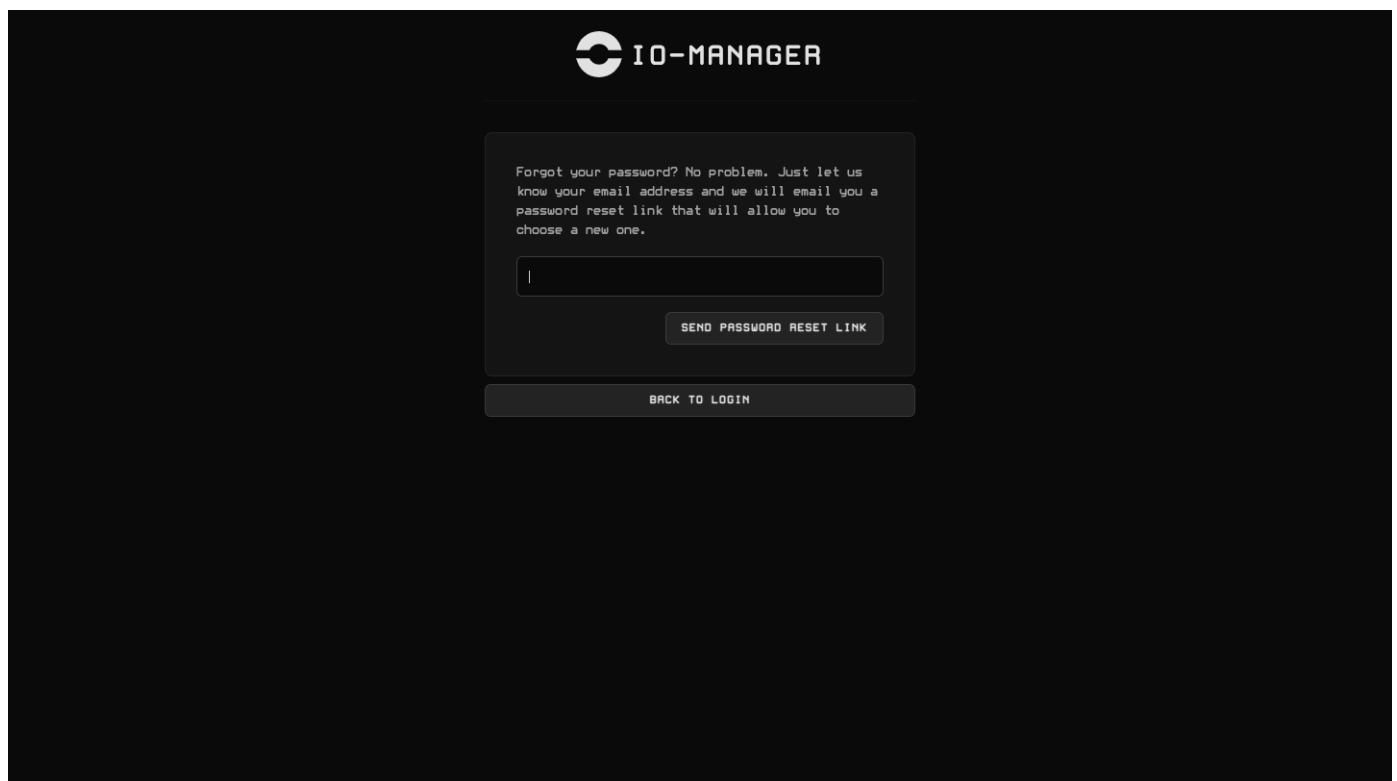
La página de inicio es básica y funcional. Aquí se empieza a entender el estilo de la aplicación, así como sus características. Permite login, crear nueva cuenta y conexión en local. El modo local permite la conexión por websockets con la placa del instrumento, pero perderá todas las funcionalidades de almacenamiento en la nube.

Además, las funciones de recordar sesión (remember me) y contraseña olvidada están activas y 100% funcionales.



The image shows a dark-themed web interface for 'IO-MANAGER'. At the top center is the logo, which consists of a white circle with a stylized 'C' inside, followed by the text 'IO-MANAGER' in a bold, sans-serif font. Below the logo is a registration form. The form is a light gray rectangle with rounded corners. It contains five input fields, each with a label above it: 'NAME', 'EMAIL ADDRESS', 'PASSWORD', 'CONFIRM PASSWORD', and 'CREATE ACCOUNT'. The 'CREATE ACCOUNT' field is a button. Below the form is a button labeled 'BACK TO LOGIN'.

Aquí se ve desplegado el formulario crear nueva cuenta.



The image shows a dark-themed web interface for 'IO-MANAGER'. At the top center is the logo, which consists of a white circle with a stylized 'C' inside, followed by the text 'IO-MANAGER' in a bold, sans-serif font. Below the logo is a password reset form. The form is a light gray rectangle with rounded corners. It contains a paragraph of text: 'Forgot your password? No problem. Just let us know your email address and we will email you a password reset link that will allow you to choose a new one.' Below the text is a single input field for the email address. Below the input field is a button labeled 'SEND PASSWORD RESET LINK'. Below the form is a button labeled 'BACK TO LOGIN'.

Aquí el formulario para email de reset de password.

2.2.2 Profile

The screenshot displays the 'IO-MANAGER' web interface. The top navigation bar includes the logo, links for 'IO-CONTROL', 'IO-CLOUD', and 'IO-FIRMWARE', and user options: '[?] ABOUT [REGULAR USER] LOGOUT'. The main content area is divided into three sections:

- Profile Information**: A section for updating account details. It includes a 'NAME' field with the value 'Regular User' and an 'EMAIL' field with the value 'user@u.com'. A 'SAVE' button is located at the bottom of this section.
- Update Password**: A section for changing the password. It contains three input fields: 'CURRENT PASSWORD', 'NEW PASSWORD', and 'CONFIRM PASSWORD'. A 'SAVE' button is at the bottom.
- DELETE ACCOUNT**: A section with a warning message 'ONCE DELETED, ALL DATA WILL BE PERMANENTLY REMOVED.' and a 'DELETE ACCOUNT' button.

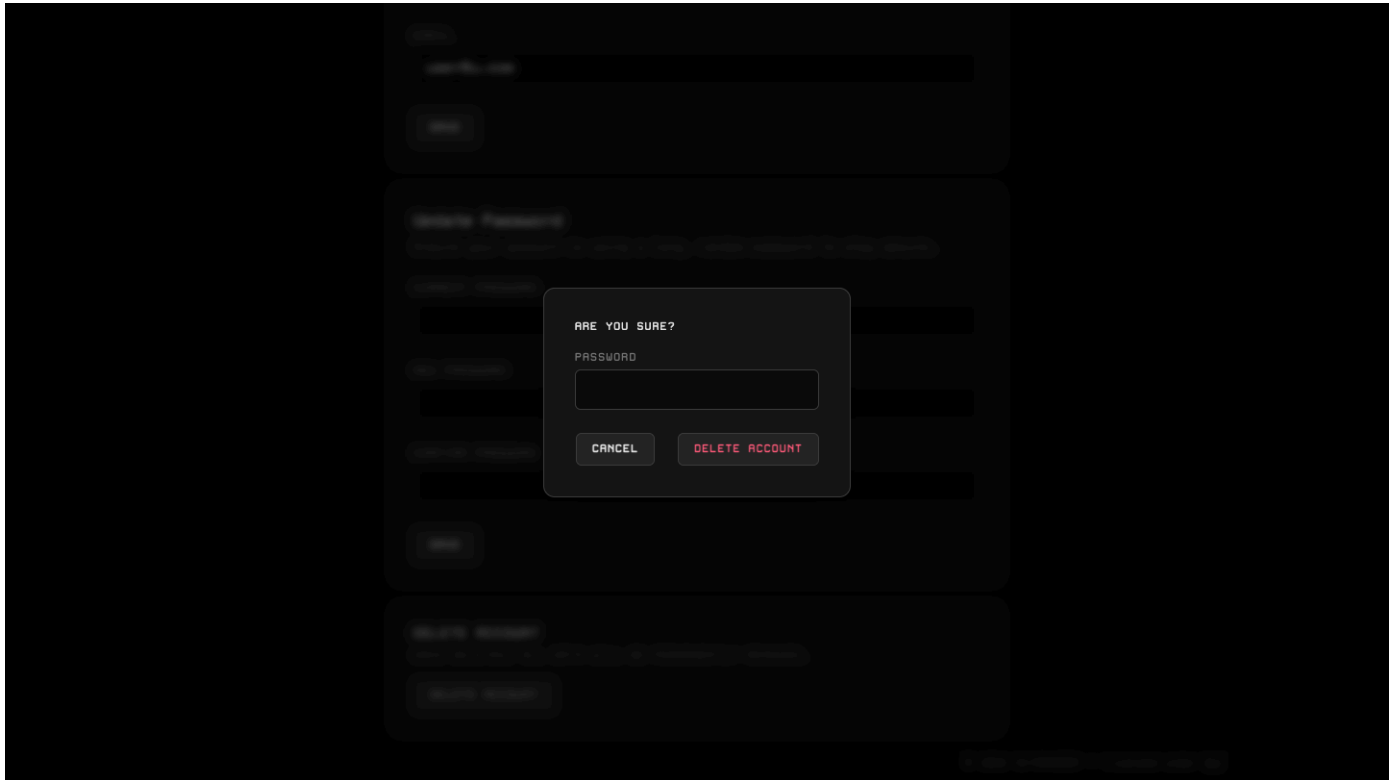
At the bottom right of the page, there is a footer: '© 2026 IO-MANAGER — Licensed under MIT'.

Una vez autenticados, podemos acceder a las funcionalidades de cuenta.

Profile information permite Modificar el nombre de usuario y el email, posteriormente a la verificación del mismo.

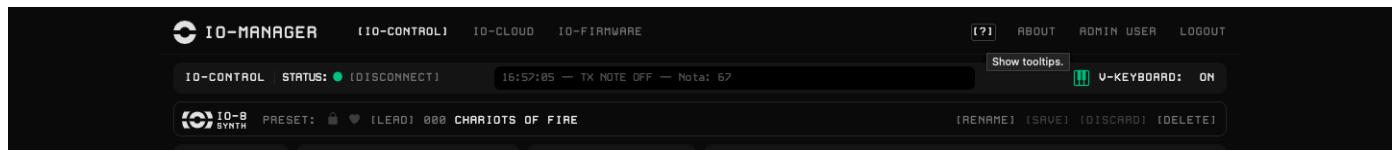
Update password también hace llamada a BD vía CORS una vez los datos se hayan introducido correctamente.

Delete account levanta un modal de confirmación en el que pide contraseña para borrar la cuenta como medida de seguridad.



Detalle del modal de eliminación de cuenta.

2.2.3 Nav, Status bar y Presets bar



Una vez autenticados podemos ver en la parte superior de la App los contextos principales.

1. **Nav bar:** La barra de navegación contiene las distintas páginas de la app.

IO-Control: Mostrará los controles principales para la modificación del sonido que se comunican bajo protocolo midi vía websockets con la placa.

IO-Cloud: Es el apartado que ofrece la funcionalidad de guardado de presets en la nube. Establece las conexiones y la sincronización con la base de datos y lista los presets guardados en repositorio local (creados o guardados por el usuario) o guardados en repositorio público (disponibles para descargar y subir al instrumento).

IO-Firmware: Ofrece conexión vía USB-Serial con la placa y establece el sistema para cargar nuevo firmware en la ESP32.

[?]: Es un toggle que permite al usuario activar y desactivar los tooltips. Si está activado, al mantener el ratón sobre cualquier elemento interactivo de la pantalla, un texto explicativo aparecerá a los pocos segundos.

About: RRSS, mail, FAQ (no implementado) y manifiesto de la App.

Profile: El navlink de profile aparece con el nombre del usuario que se haya autenticado, permite entrar en las opciones de cuenta.

Logout: Desautenticarse. En el modo local, este botón queda renombrado como Exit. En ambos casos, lleva al usuario de vuelta a Landing page.

2. **Status bar:** Es un contexto al que pertenecen las páginas IO-Control, IO-Cloud e IO-Firmware (en esta última V-Keyboard estará siempre desactivado).

Status: Muestra el estado de la conexión websocket con el microcontrolador.

Midi monitor: Muestra los eventos midi generados tanto por los controles como por V-Keyboard.

V-Keyboard: Convierte el teclado del pc en un controlador MIDI de dos octavas, con control de velocidad (teclas 9 y 0) y control de octava (teclas ' y j).



3. **Presets bar:** Presets bar estará activa y visible en IO-Control y en IO-Cloud, desapareciendo del contexto en el resto de páginas de la App. Es la encargada del management y selección de los presets del instrumento. En esta imagen se puede ver el preset cargado y la lista de presets desplegada.

Lock (icono del candado): Levanta el flag de readOnly del preset y desactiva el botón de Delete en el preset seleccionado.

Fav (icono del corazón): Levanta el flag de Favorite en el preset.

Slot + nombre: Indica el número de slot en el que está guardado el preset, su nombre y tiene funcionalidad toggle para desplegar la lista de presets.

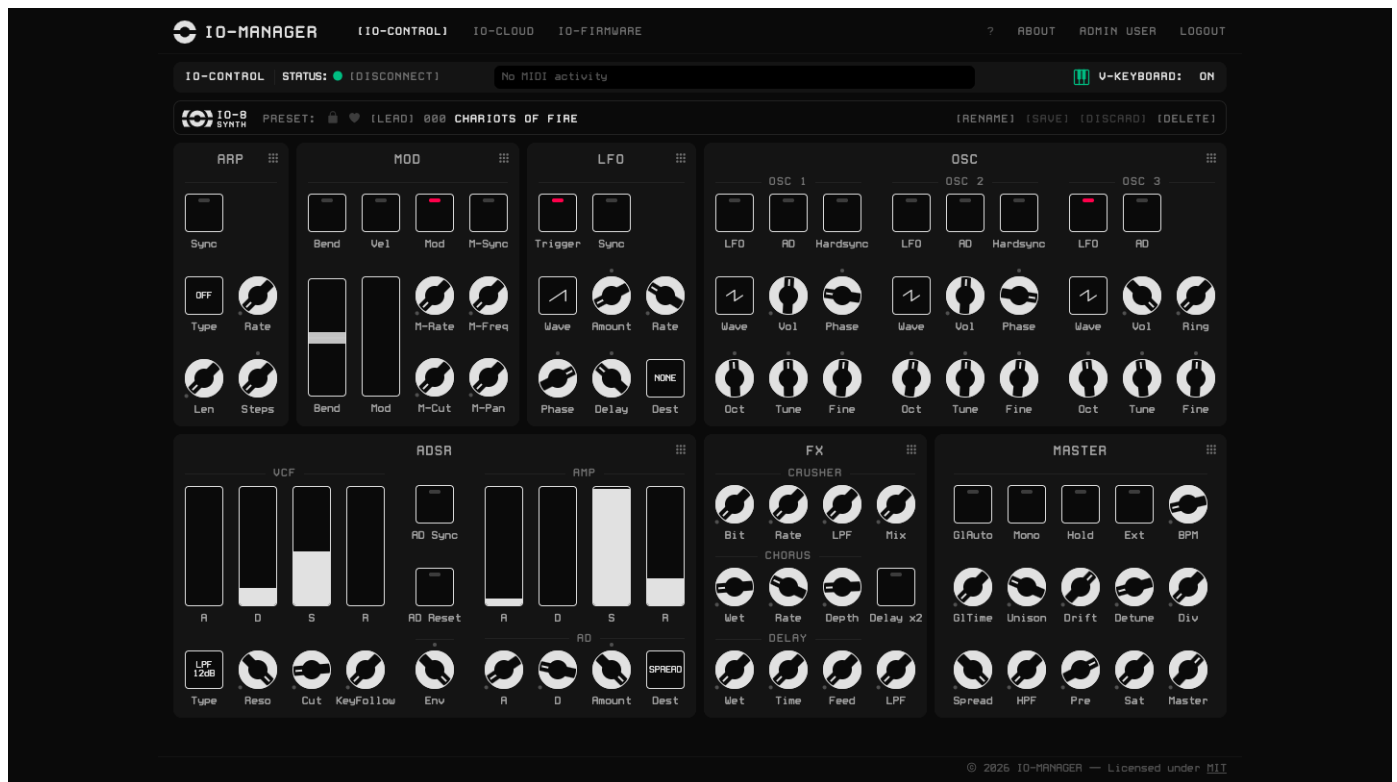
Rename: Despliega un modal que permite cambiar el nombre del preset.

Save: Guarda el nuevo preset, o los cambios realizados sobre un preset existente.

Discard: Descarta los cambios del preset y lo devuelve a su último snapshot.

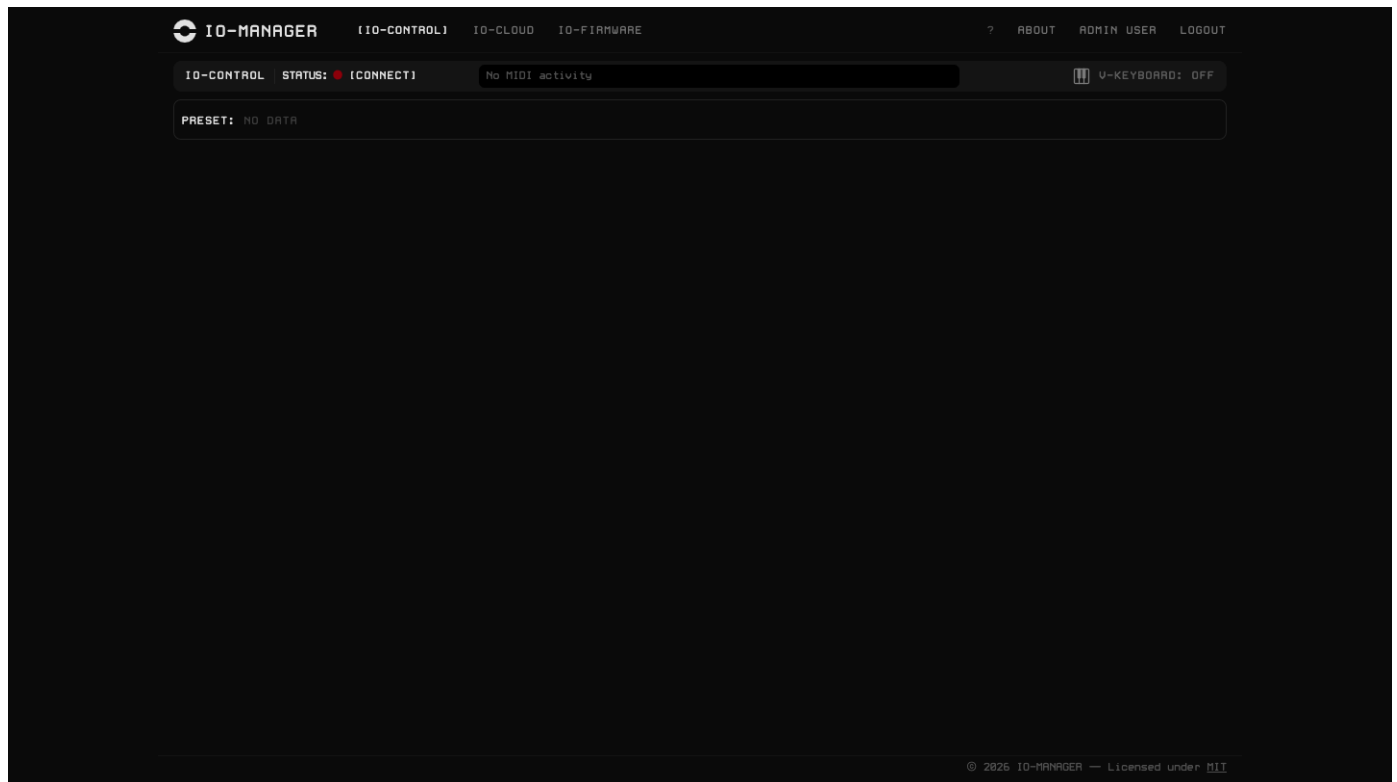
Delete: Levanta modal de confirmación y permite borrar el preset seleccionado del instrumento. Este botón de delete es el único que puede borrar un preset del dispositivo.

2.2.4 IO Control



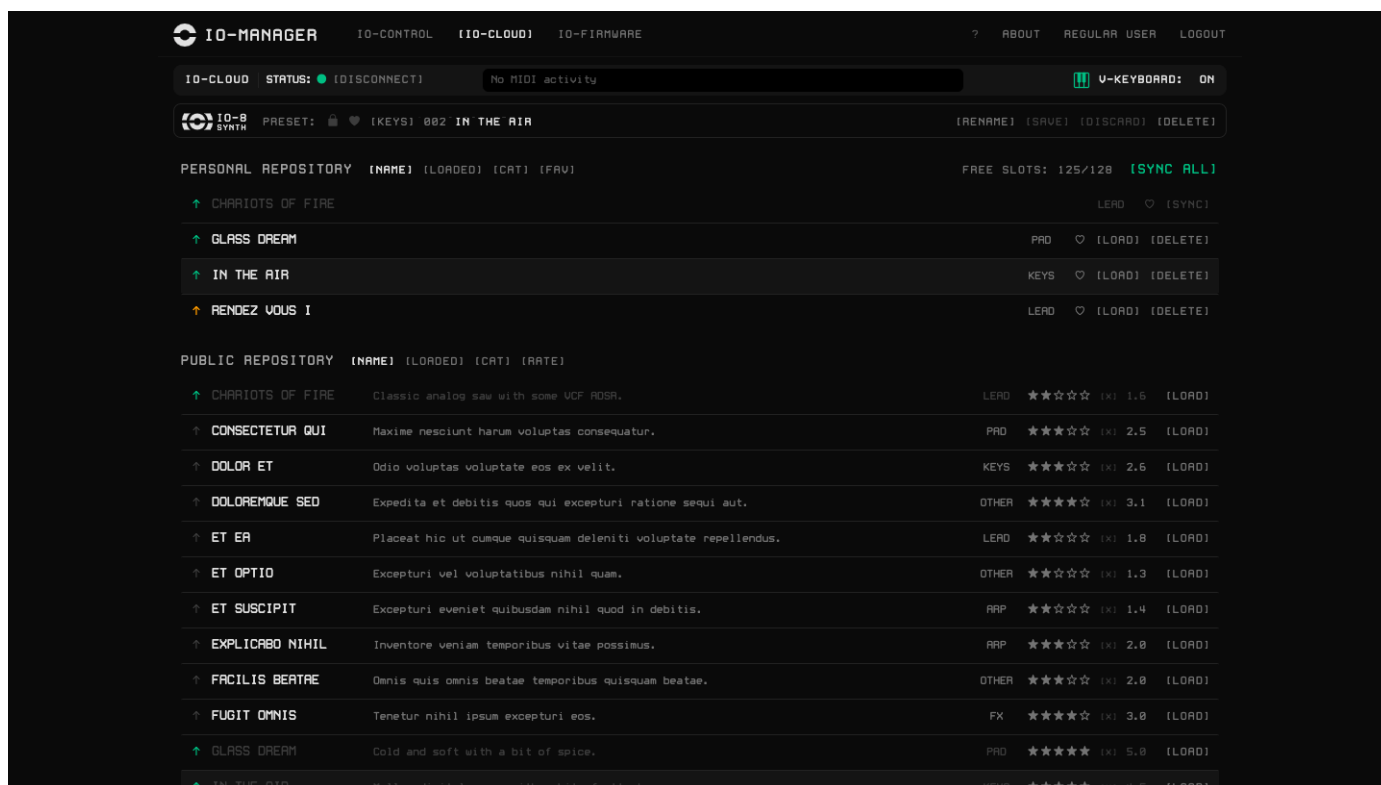
En el momento en el que el dispositivo se conecta por websocket, IO-Control despliega los módulos de los controles dedicados para manejar los parámetros del instrumento modificando el preset que se encuentre cargado. La activación de cualquiera de estos controles dispara el protocolo MIDI CC (Control Command) correspondiente hacia la placa.

Estos módulos tienen drag and drop, son adaptativos y se pueden reorganizar y reapiilar al gusto del usuario.



Si el instrumento está apagado o no está estableciendo conexión por websocket, IO Control se verá así. Preset mostrará "no data" y los controles asociados con el instrumento desaparecerán del panel.

2.2.5 IO Cloud



La funcionalidad de IO-Cloud va a diferir en base a si el usuario entra sin autenticar (modo local) y en base al rol de la cuenta.

Al entrar en modo local, la funcionalidad de Personal repository queda relegada a listar los presets cargados en el microcontrolador y Public repository desaparece por completo.

Con un usuario autenticado, las opciones de administración quedarán inactivas y no se renderizarán, pero el usuario tendrá acceso al repositorio privado y público, para sincronizar sus presets con el dispositivo y su cuenta en la nube.

Un usuario con rol Admin podrá publicar y borrar presets en el repositorio público.

Sorting buttons: En Personal Repository la lista de presets se puede ordenar en base a nombre (asc - desc), a si está cargado en los slots del instrumento, en base a su categoría (recorrido en loop) y a si el flag favorite está seleccionado. En Public Repository name y loaded y cat hacen lo mismo y se añade rate. Rate ordena primero los presets votados (panel Stars) y después los no votados, ambos en ascendente y descendente.

Free Slots: Indicador del número de slots libre en el instrumento. En caso de no quedar slots libres, la aplicación deshabilitará el botón de load tanto en el repositorio privado como en el público.

Indicadores colores y arrow up: El indicador de flecha marca en verde si el preset está subido al dispositivo y en amarillo si se encuentra en la base de datos local pero no en el dispositivo. Si el preset está subido al dispositivo pero no está en la base de datos personal, la línea del preset aparecerá en gris oscuro y se habilitará el botón de Sync. El preset cargado en la placa tendrá un highlight en gris en el background tanto en el repositorio privado como en el repositorio público si existe.

Sync: Si el preset cargado en el dispositivo no se encuentra sincronizado con la base de datos privada, el botón Sync se activará (marcado en verde). Al pulsarlo se activa el empaquetamiento del preset y se sincroniza.

Sync All: Si existe más de un preset cargado en el instrumento pero no sincronizado con Personal Repository, este botón hará un barrido cargando, empaquetando y subiendo los presets a la base de datos.

Load: Permite cargar el preset en la placa en caso de que haya slots libres.

Delete: (no confundir con delete de Preset bar). Delete permite borrar el preset listado de la base de datos. Si el preset se encuentra en el dispositivo, permanecerá allí.

Stars Panel: Permite votar los presets con valores del 1 al 5 también permite eliminar el voto con [x]. En caso de haber votado un preset, el Stars panel tendrá un highlight en amarillo y mostrará el voto del usuario por encima del voto promediado. Al lado siempre se mostrará la media aritmética de todos los votos de los usuarios en valor decimal.

IO-MANAGER IO-CONTROL IO-CLOUD IO-FIRMWARE [?] ABOUT ADMIN USER LOGOUT

IO-CLOUD STATUS: [DISCONNECT] 16:57:05 — TX NOTE OFF — Nota: 67 U-KEYBOARD: ON

IO-8 SYNTAX PRESET: [PAD] 001 GLASS DREAM [RENAME] [SAVE] [DISCARD] [DELETE]

PERSONAL REPOSITORY [NAME] [LOADED] [CAT] [FAV] FREE SLOTS: 126/128 [SYNC ALL]

↑ CHARIOTS OF FIRE LEAD [SYNC]

↑ GLASS DREAM PAD [LOAD] [DELETE] [PUBLISH]

↑ IN THE AIR KEYS [LOAD] [DELETE] [PUBLISH]

↑ RENDEZ VOUS I LEAD [LOAD] [DELETE] [PUBLISH]

↑ RENDEZ VOUS II LEAD [LOAD] [DELETE] [PUBLISH]

PUBLIC REPOSITORY [NAME] [LOADED] [CAT] [RATE]

↑ CHARIOTS OF FIRE Classic analog saw with some VCF ADSR. LEAD ★★★★★ (x) 1.6 [LOAD] [DELETE]

↑ CONSECTETUR QUI Maxime nesciunt harum voluptas consequatur. PAD ★★★★★ (x) 2.5 [LOAD] [DELETE]

↑ DOLOR ET Odio voluptas voluptate eos ex velit. KEYS ★★★★★ (x) 2.6 [LOAD] [DELETE]

↑ DOLOREMQUE SED Expedita et debitis quos qui excepturi ratione sequi aut. OTHER ★★★★★ (x) 3.1 [LOAD] [DELETE]

↑ ET EA Placeat hic ut cumque quisquam deleniti voluptate repellendus. LEAD ★★★★★ (x) 1.8 [LOAD] [DELETE]

↑ ET OPTIO Excepturi vel voluptatibus nihil quam. OTHER ★★★★★ (x) 1.3 [LOAD] [DELETE]

↑ ET SUSCIPIT Excepturi eveniet quibusdam nihil quod in debitis. ARP ★★★★★ (x) 1.4 [LOAD] [DELETE]

↑ EXPLICABO NIHIL Inventore veniam temporibus vitae possimus. ARP ★★★★★ (x) 2.0 [LOAD] [DELETE]

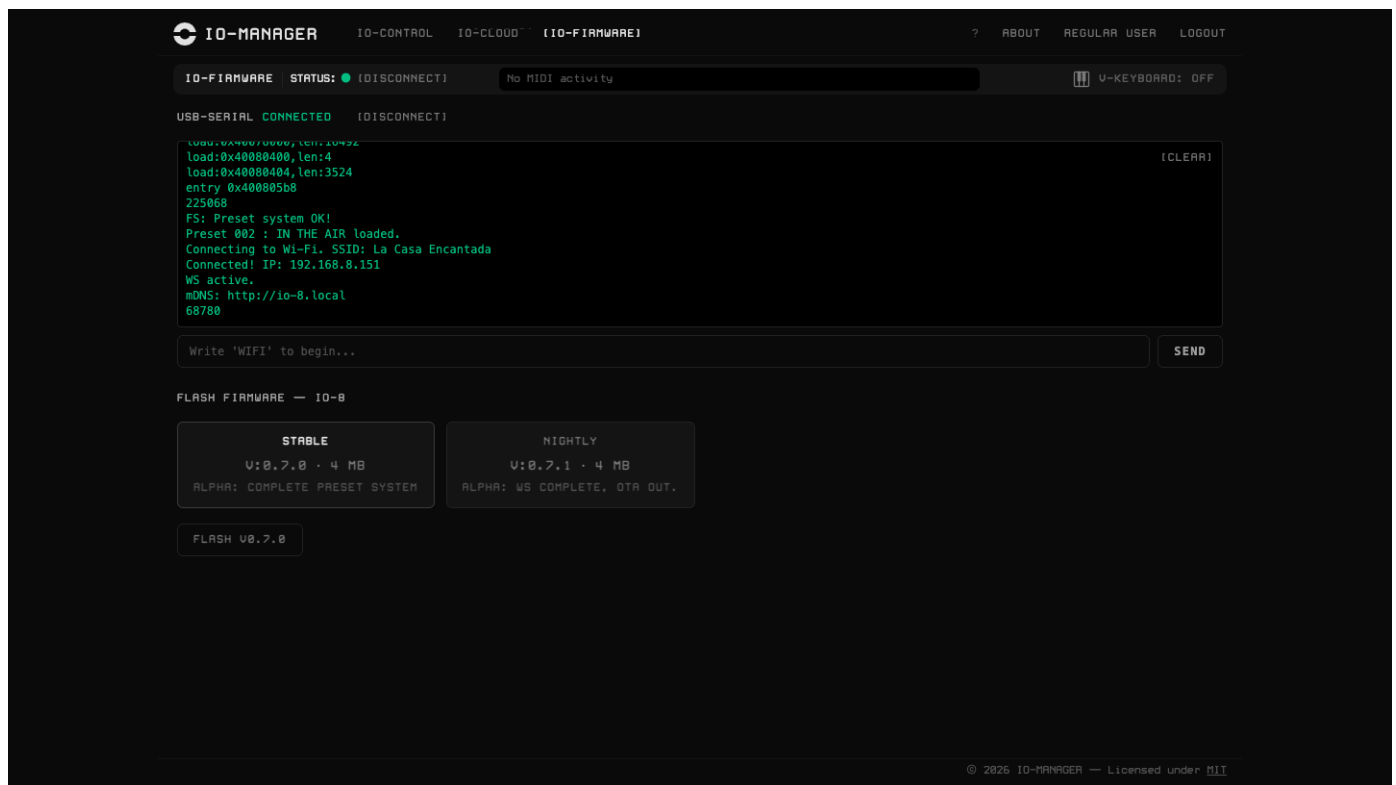
↑ FACILIS BEATAE Omnis quis omnis beatae temporibus quisquam beatae. OTHER ★★★★★ (x) 2.0 [LOAD] [DELETE]

↑ FUGIT OMNIS Tenetur nihil ipsum excepturi eos. FX ★★★★★ (x) 3.0 [LOAD] [DELETE]

↑ GLASS DREAM Fold and puff with a bit of noise. PAD ★★★★★ (x) 5.0 [LOAD] [DELETE]

Detalle de una cuenta con el rol de admin activado, se puede ver cómo aparecen los nuevos botones "Publish" en Personal Repository y "Delete" en Public Repository, lo que permite administrar la sección pública de la base de datos subiendo y borrando nuevos presets.

2.2.6 IO Firmware



IO-Firmware es la sección que se encarga de la conexión USB-Serial con la ESP32 y la carga de los últimos firmwares de los instrumentos.

Serial Monitor: En caso de conectarse mediante USB-Serial, el Serial Monitor comenzará a lanzar el contenido que la placa envíe. A mayores, para establecer las acreditaciones wifi (persistentes en la placa mediante NVS) necesaria para levantar el servidor y establecer la conexión por websockets, se ha habilitado una línea de comandos que convierte los inputs en un protocolo binario customizado (medida de seguridad) y que admite comandos como start, wifi, cancel, exit y otros, por los que la placa irá guiando al usuario para establecer dichas credenciales.

Flash Firmware: Permite seleccionar entre dos modos de firmware, Stable y Nightly, estando el segundo canal dedicado a versiones experimentales. Al presionar el botón Flash, la librería esp-tool lanzará el proceso de burn del firmware y reboot de la placa. Para el first boot, la placa tendrá que resetearse manualmente y después de ello, comenzará el protocolo de creación del file system. Posteriormente el usuario podrá comenzar con la configuración wifi en Serial monitor.

Project set up in the IO-8 board
Connecting to Wi-Fi. SSID: La Casa Encantada
Connected! IP: 192.168.8.151
WS active.
mDNS: http://io-8.local
63224

Write 'WIFI' to begin...

SEND

FLASH FIRMWARE — IO-8

STABLE
V:0.7.0 · 4 MB
ALPHA: COMPLETE PRESET SYSTEM

NIGHTLY
V:0.7.1 · 4 MB
ALPHA: WS COMPLETE, OTA OUT.

FLASH V0.7.0

UPLOAD IO-8 FIRMWARE:

FILE (.BIN)
SELECT FILE

VERSION
EJ: 0.6.0

CHANNEL
STABLE

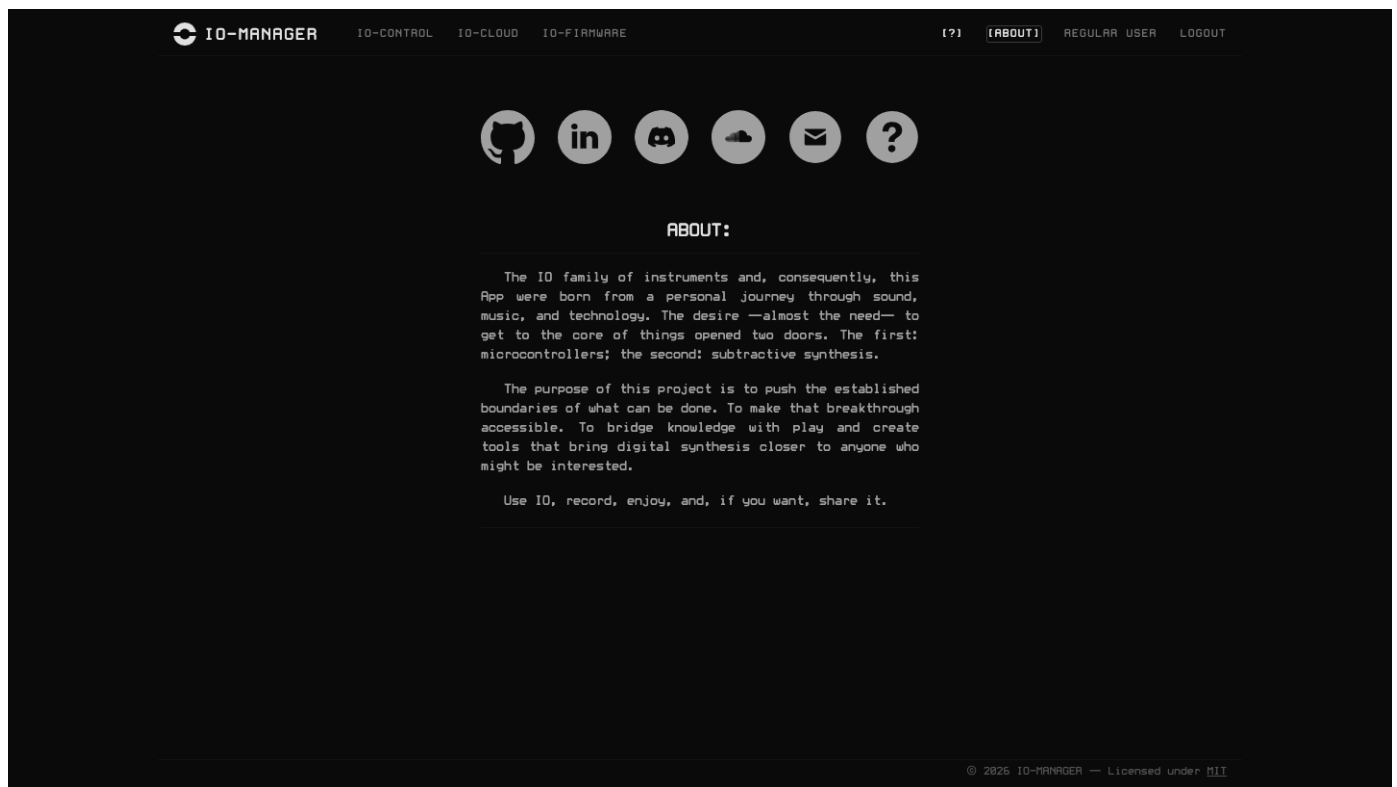
RELEASE NOTES
Cambios o correcciones...

UPLOAD FIRMWARE

© 2026 IO-MANAGER — Licensed under [MIT](#)

En caso de loguearse con rol de admin, el formulario Upload Firmware aparecerá debajo. Este formulario permite cargar los nuevos binarios a la base de datos.

2.2.7 About



About ofrece al usuario vías de contacto con el creador, RRSS y demás cuentas.

2.3 Desarrollo de la App

2.3.1 Arquitectura

La aplicación se rige por un modelo desacoplado de cliente robusto (Desktop App) y servidor centralizado (RESTful API). Originalmente concebido como un entorno monolítico basado en Laravel Breeze e Inertia.js, el sistema mutó radicalmente hacia un cliente de escritorio nativo con React, interactuando de forma asíncrona mediante la biblioteca Axios con un backend en Laravel. Esta decisión estratégica elimina las limitaciones impuestas por el aislamiento de los entornos de ejecución WebView y garantiza una óptima modularidad. El backend opera como un proveedor de servicios sin estado (stateless), encargado del almacenamiento persistente, auditorías de seguridad, autenticación y distribución controlada de binarios de firmware.

La arquitectura frontend de IO-Manager implementa una versión adaptada de Domain-Driven Design (DDD) combinada con una estructura orientada a características (Feature-Driven). En lugar de agrupar los archivos estrictamente por su tipo técnico (todos los hooks juntos, todos los componentes juntos), el sistema se segmenta por subdominios de negocio dentro del directorio Features/Device/. Esta segregación garantiza un acoplamiento débil (loose coupling) y una alta cohesión, permitiendo que cada módulo sea autónomo y escalable. El proyecto utiliza un patrón de Composición de Layouts Anidados, lo que optimiza el rendimiento del Virtual DOM al evitar re-renderizados innecesarios de las estructuras comunes de la interfaz durante la navegación. El subdominio Shared provee la infraestructura reactiva y lógica que interconecta el hardware nativo con la interfaz gráfica de React.

Se centraliza el estado de comunicación bidireccional mediante un contexto de React (WsContext.jsx). Este enfoque mitiga el prop drilling y permite que distintas vistas expongan o reaccionen al estado del instrumento en tiempo real. uUeWebSocket.js abstrae la lógica del protocolo WebSocket. Implementa el temporizador supervisor (connTimeoutRef) y la lógica de reintentos asíncronos exponiendo estados limpios (Connected, Disconnected, Reconnecting). Mientras, wsMsgHandle.js: Actúa como un despachador puro de comandos (command dispatcher). Procesa las ráfagas de bytes entrantes de la conexión y las traduce en actualizaciones de estado tipadas que React puede interpretar de forma eficiente.

Se inyectaron configuraciones directas de sockets TCP (SO_KEEPALIVE, TCP_KEEPIIDLE, TCP_KEEPIIDLE) complementadas con un temporizador supervisor (connTimeoutRef) en el cliente que aborta la conexión en 2 segundos si el canal físico experimenta degradación, inician ciclos controlados de reconexión con reporte visual de estado.

Dado a que se requieren interfaces de comunicación sólidas de red y de hardware. Para mitigar la latencia y evitar la saturación de datos (buffer overrun) en ráfagas de notas MIDI o movimientos de potenciómetros masivos, se diseñó un protocolo de Control de Flujo por Lotes (Batch Processing Flow Control) sobre la capa de WebSockets. El frontend acumula las

transmisiones en un búfer local. La aplicación detiene el envío hasta que recibe un frame de control. En ese instante, la aplicación pone el flag `espReadyRef.current = true`, vacía el búfer pendiente (flush) y permite la siguiente ráfaga de datos, logrando una comunicación asíncrona de tiempo real sin bloqueos.

La aplicación maneja tres capas complejas de información en tiempo real: los metadatos de la memoria física del instrumento, la base de datos local y el repositorio en la nube. El guardado de presets implementa una máquina de estados sumamente estricta que evita pérdidas accidentales de información. Al cargar un preset, se congela un clon binario de sus parámetros en una referencia mutable (`snapshotRef`). El botón de guardado y los componentes gráficos analizan constantemente el estado actual frente a la instantánea. Si el usuario modifica los valores del sintetizador pero los devuelve manualmente a su posición original, el indicador binario `modified` se desactiva de forma automática.

El sistema implementa dos roles diferenciados de usuario. La seguridad perimetral de los endpoints administrativos (como la publicación de firmwares globales o la moderación del repositorio público de presets) está blindada mediante el Middleware, inyectado directamente en el flujo de arranque del núcleo de la aplicación (`bootstrap/app.php`).

Se unificaron los motores lógicos `usePrivateRepo` y `usePublicRepo` en una única abstracción con el fin de unificar el ordenamiento avanzado (Sorting). El motor de ordenamiento evalúa en milisegundos las listas bajo criterios dinámicos

El hook de bajo nivel `useSerial.js` se refactorizó para gestionar los estados de cierre del puerto mediante promesas explícitas (`closingPromise`), erradicando las colisiones de hilos de ejecución cuando se conmuta de página a alta velocidad.

Para automatizar la entrada en modo de carga de binarios, el módulo de flasheo ejecuta de forma síncrona comandos sobre las líneas físicas de control de flujo serie (DTR y RTS), implementando retardos críticos de estabilización para alternar los estados del chip.

El panel de control (Control/) rompe con las interfaces estáticas mediante una arquitectura modular basada en el ecosistema `@dnd-kit/core`. Los módulos funcionales del instrumento (`OscModule`, `LfoModule`, `AdsrModule`, `FxModule`, etc.) están desacoplados de su contenedor base `ModuleGrid.jsx`. La persistencia de la posición física en la cuadrícula se calcula de manera asíncrona mediante utilidades de ordenación (`sortable`), permitiendo una reconfiguración completa en caliente de la UI.

El subdominio Cloud unifica la persistencia de datos local (memoria del dispositivo) y remota (base de datos de la comunidad). Abstracción y Unificación (`RepoWrapper.jsx` y `useRepo.js`): Tras la unificación de los repositorios público y privado, `RepoWrapper.jsx` actúa como un componente de orden superior que renderiza listas de presets bajo el mismo estándar visual (`Repo.jsx`).

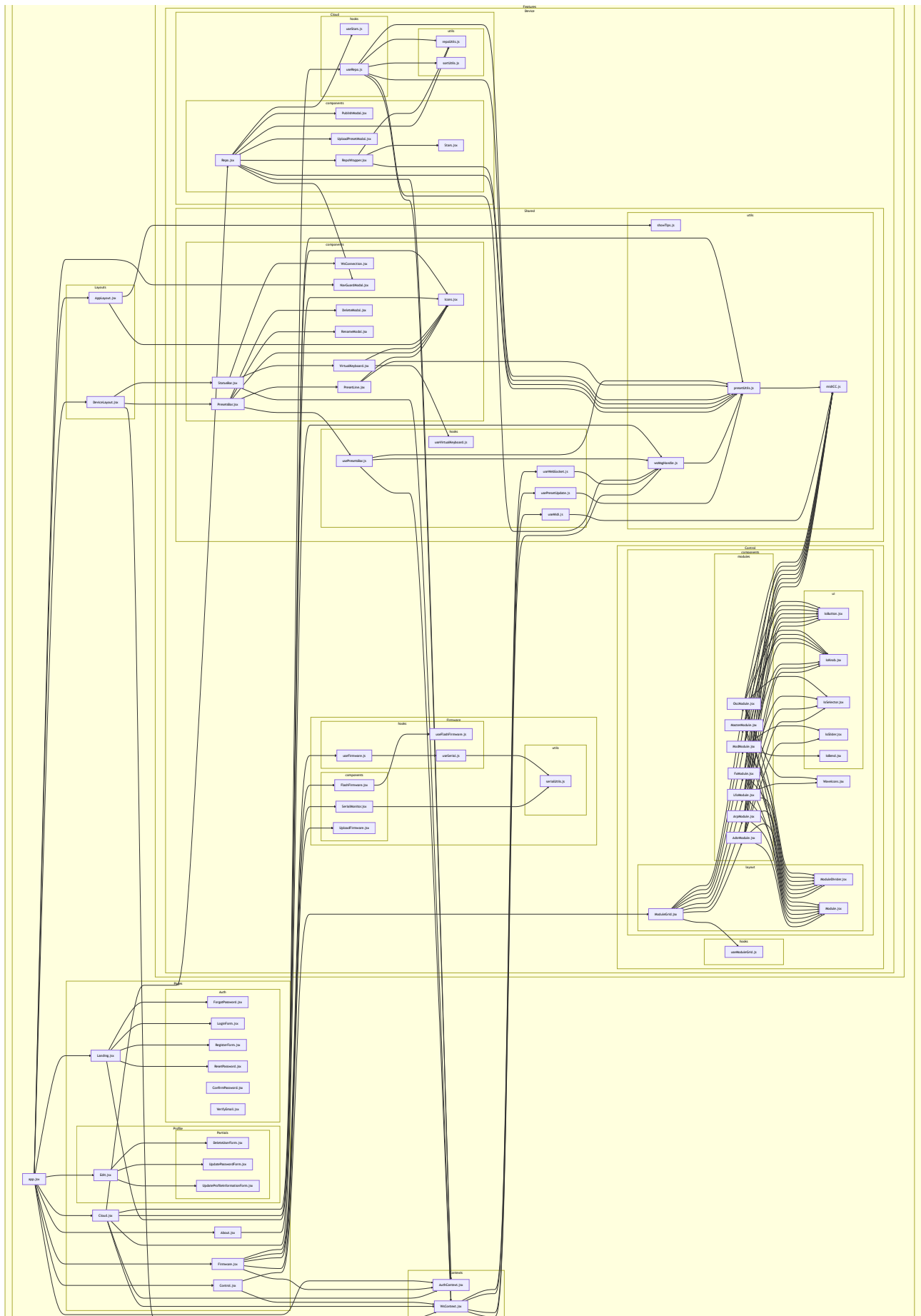
Diagrama general frontend: (ampliar para ver el detalle).

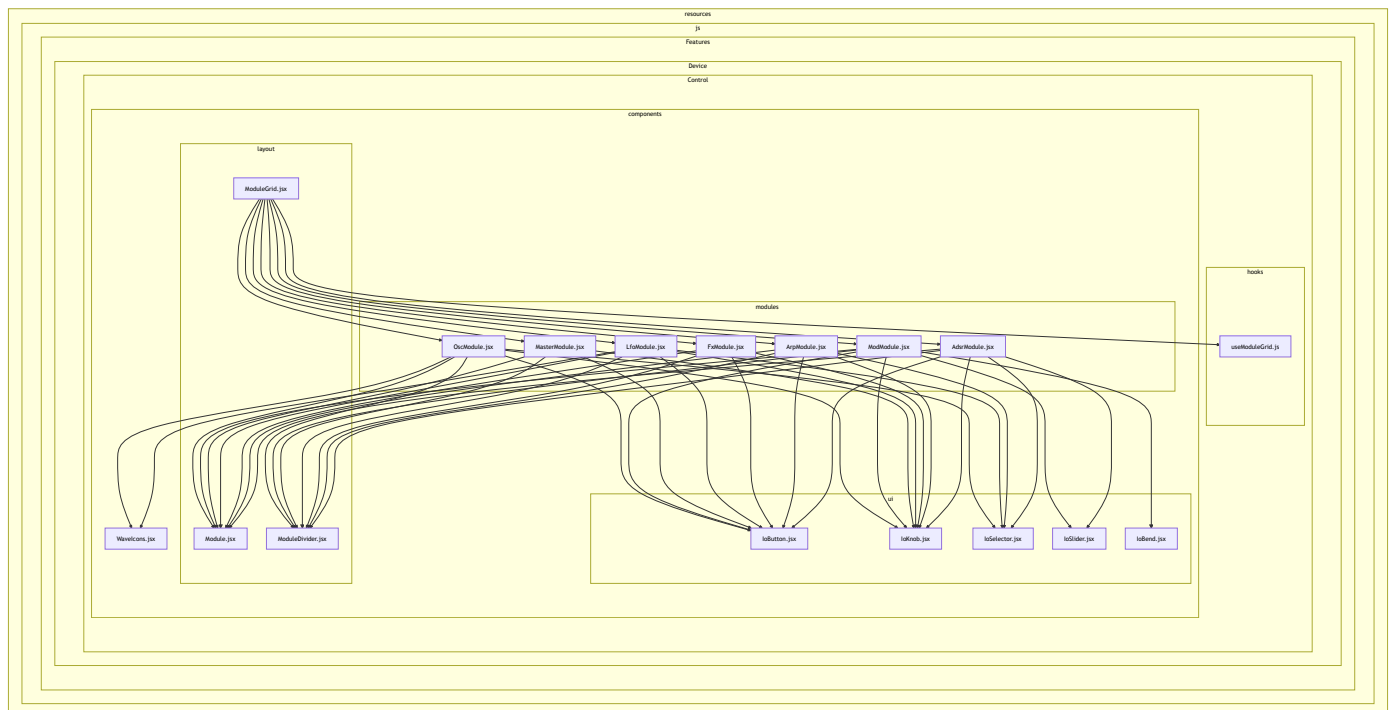
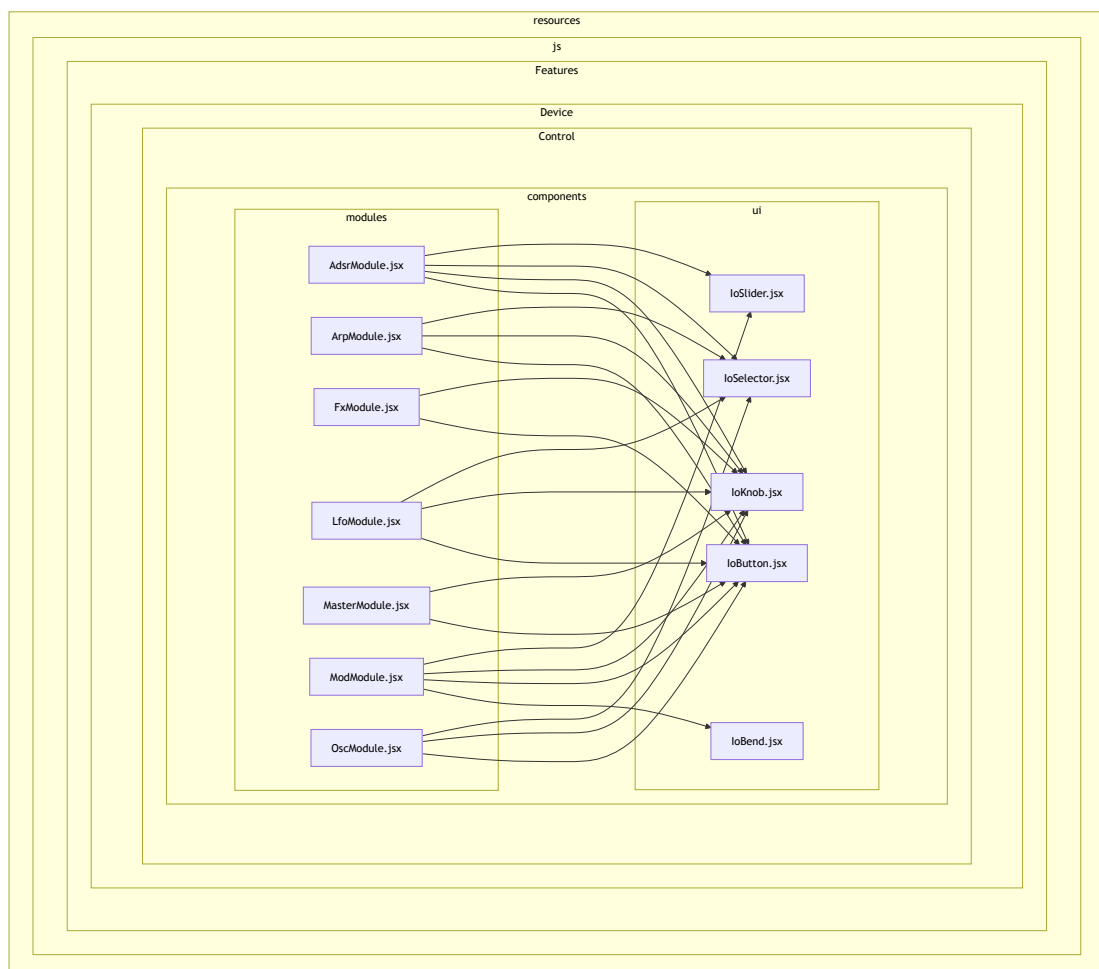
Diagrama Control: (ampliar para ver el detalle).Diagrama Control (detalle módulos):

Diagrama Cloud:

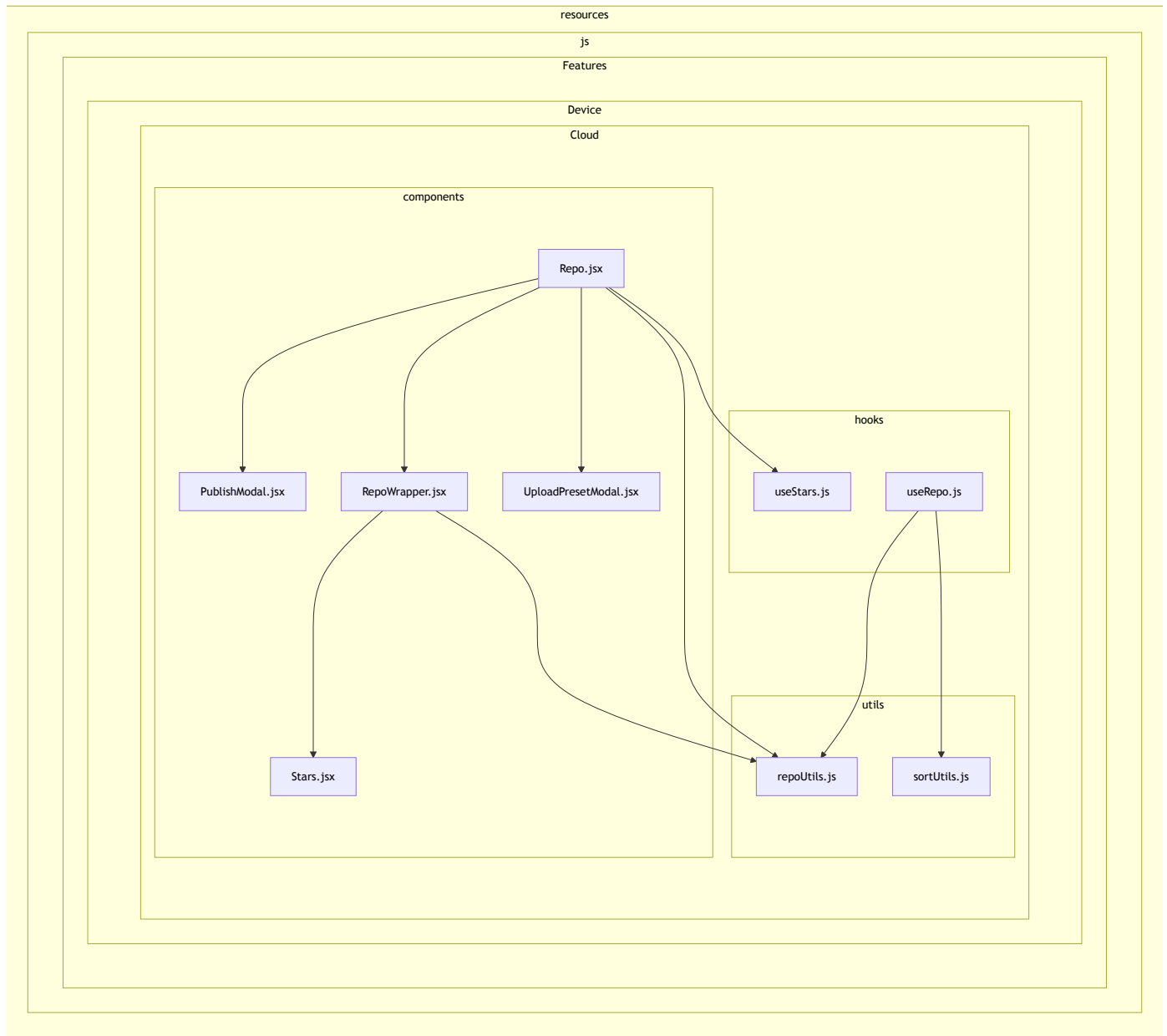


Diagrama Firmware:

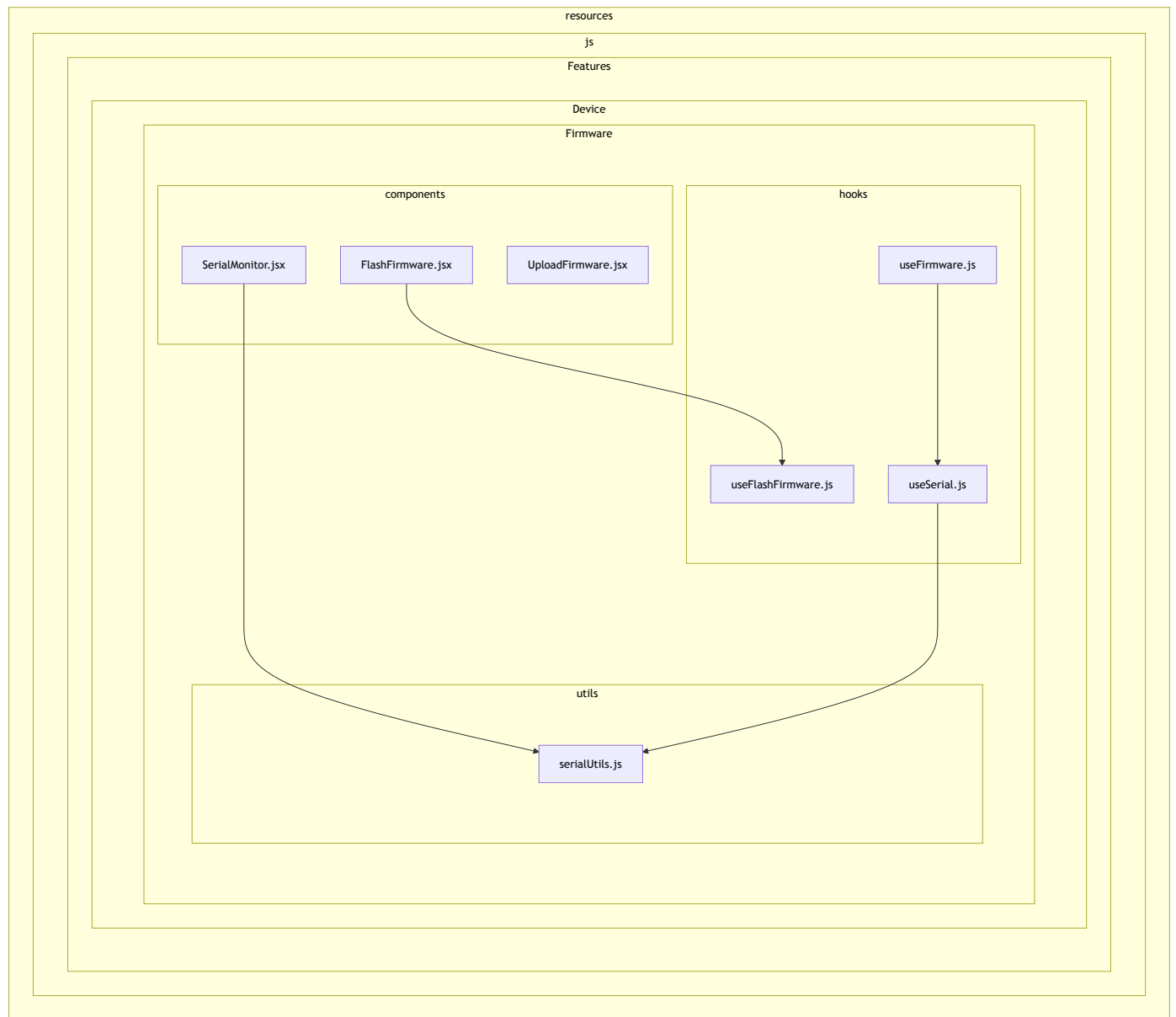
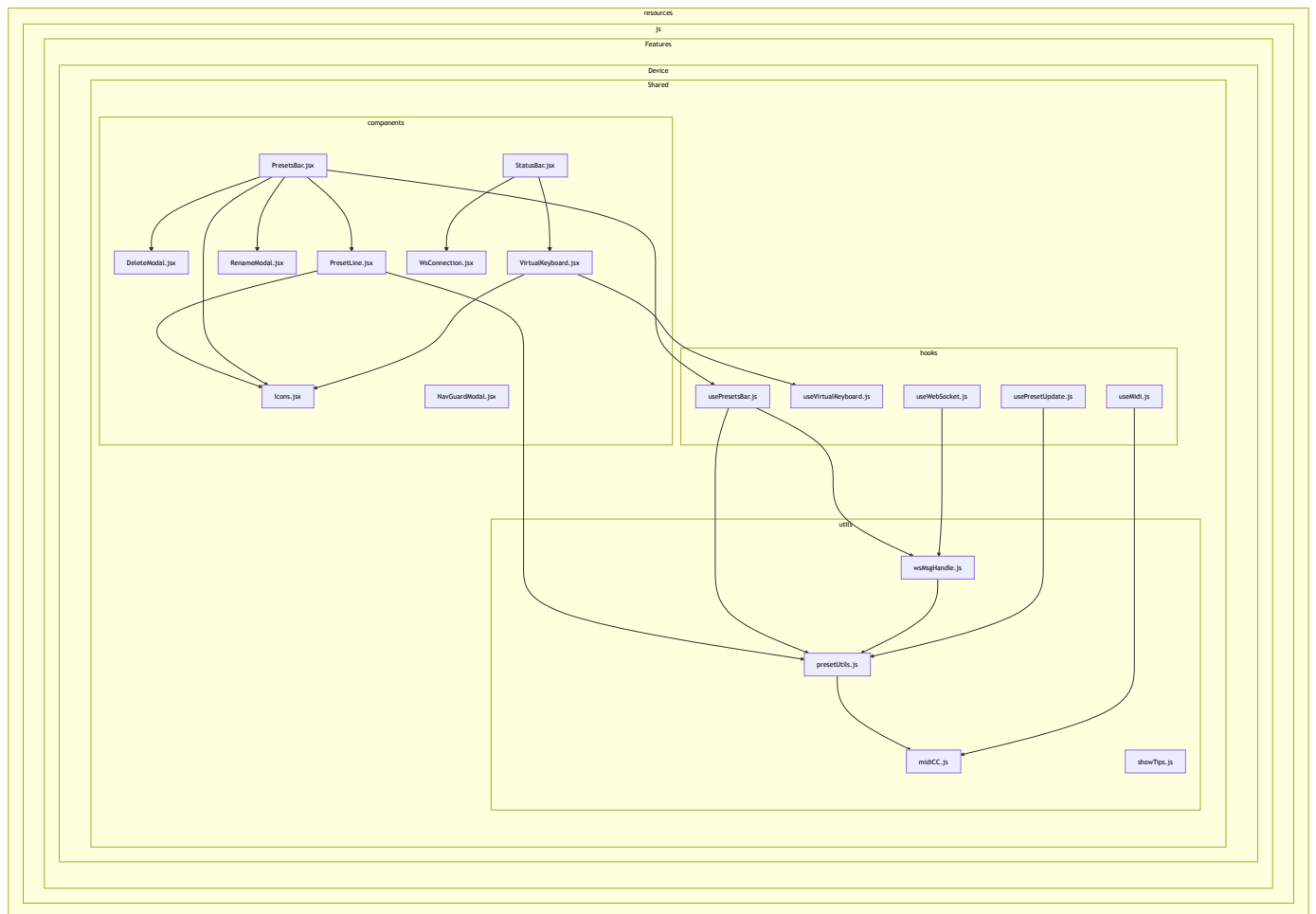
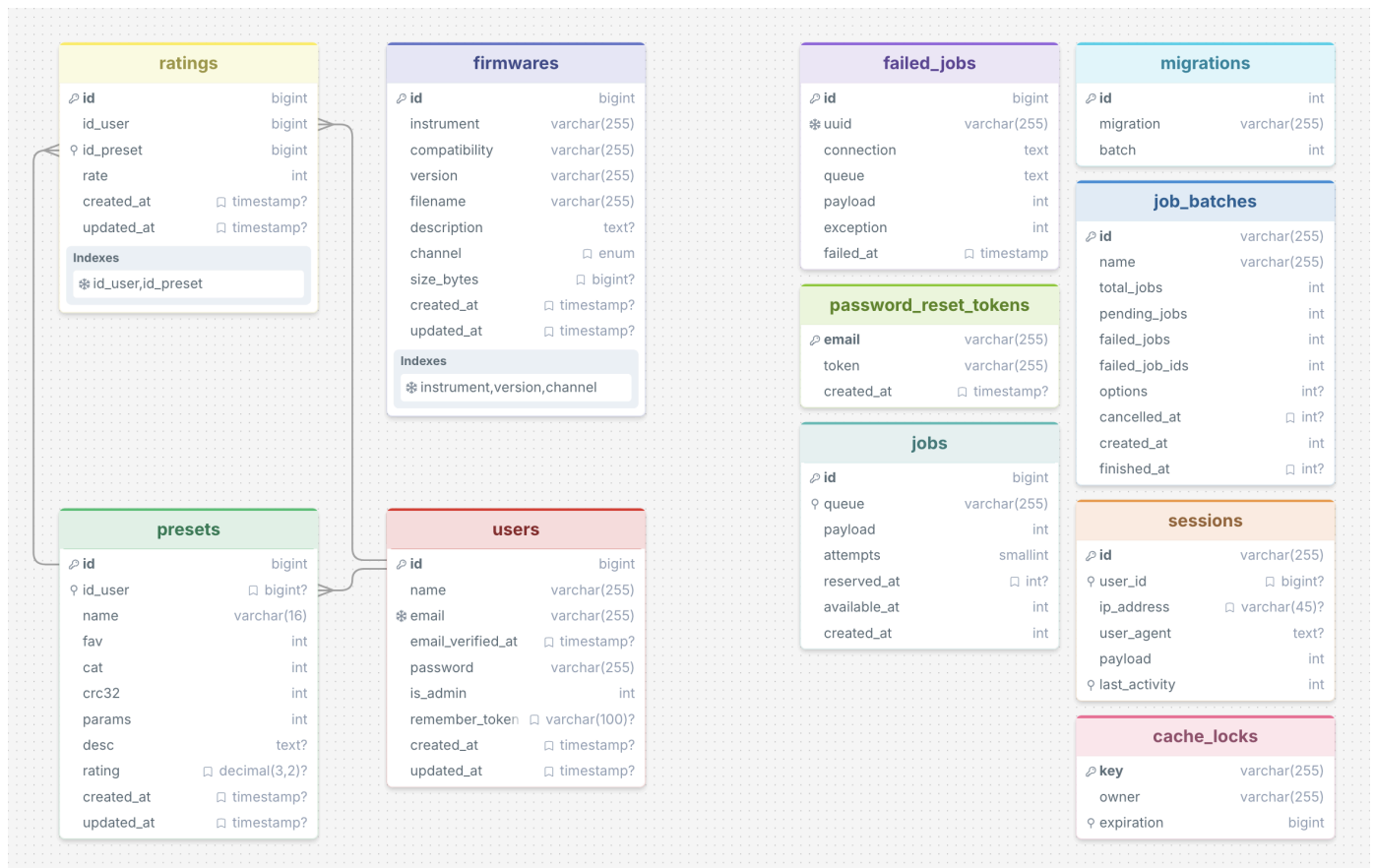


Diagrama Shared: (ampliar para ver el detalle).



2.3.2 Base de datos



Entidades Principales

Users: Almacena la identidad, credenciales y define el nivel de acceso del middleware mediante el booleano is_admin.

Presets: Contiene las configuraciones del instrumento (params en varbinary(128)), clasificación (cat), integridad matemática (crc32) y el promedio de calificación (rating).

Ratings: Tabla transaccional pura que almacena la puntuación numérica (rate) otorgada a una configuración.

Firmwares: Administra los archivos de sistema (filename), instrumento destino, versión y canal de distribución (stable o nightly).

Relaciones y Restricciones

Users (1) → (N) Presets: Un usuario puede crear múltiples configuraciones. La clave foránea id_user en la tabla presets permite valores nulos; si es null, el sistema asume que es un preset global o de fábrica. Cuenta con borrado en cascada (ON DELETE CASCADE).

Users (1) → (N) Ratings: Un usuario puede realizar múltiples votaciones. La clave foránea `id_user` tiene borrado en cascada.

Presets (1) → (N) Ratings: Un preset puede acumular múltiples puntuaciones. La clave foránea `id_preset` tiene borrado en cascada.

Users (N) → (M) Presets (Vía Ratings): La tabla ratings actúa como entidad pivote para resolver la relación muchos a muchos. Utiliza un índice único compuesto `UNIQUE(id_user, id_preset)` para garantizar matemáticamente que un usuario solo pueda votar una única vez por cada preset.

Firmwares (Aislada): Opera sin claves foráneas, pero emplea un índice único compuesto `UNIQUE(instrument, version, channel)` para evitar la duplicación de binarios en el mismo canal de actualización.

El modelo relacional se articula en torno a cuatro entidades: usuarios, presets, valoraciones y firmware.

La tabla de usuarios gestiona la autenticación mediante Sanctum y define el nivel de privilegio a través de un atributo booleano `is_admin`, que distingue los perfiles administrativos de los estándar.

Los presets se almacenan en un campo binario de 128 bytes que encapsula los parámetros del sintetizador en crudo. La integridad de cada registro se delega a una columna `CRC32` calculada sobre dichos parámetros. La relación con el usuario es opcional: un preset con `id_user` nulo se interpreta como preset de fábrica. Las valoraciones se modelan como una entidad pivote entre usuarios y presets, con una restricción de clave única compuesta (`id_user, id_preset`) que impide la duplicidad de votos por cuenta.

La tabla de firmware está aislada del resto del modelo y protegida por un índice único sobre la combinación (`instrument, version, channel`), que previene colisiones en el catálogo de distribución.

La lógica de negocio se implementa mediante controladores con validación formal y control de acceso por roles. La subida de binarios y la escritura de metadatos de firmware están restringidas a perfiles con privilegios administrativos verificados en sesión. Los endpoints de presets segmentan las consultas entre el repositorio público y las colecciones privadas, y aplican guardas de autorización que limitan las operaciones de escritura y borrado al autor del contenido o a un administrador.

El recálculo del rating medio se gestiona mediante observadores del ciclo de vida del modelo, que actualizan la columna `rating` de la tabla de presets de forma síncrona tras cada creación o eliminación de una valoración.

Todas las relaciones foráneas están definidas con borrado en cascada, garantizando que la eliminación de una cuenta de usuario elimine también su contenido asociado sin dejar registros huérfanos.

El entorno de desarrollo se inicializa mediante factories y seeders que instancian un contexto determinista: un perfil administrador, un usuario estándar de pruebas, un conjunto de usuarios generados aleatoriamente, y un catálogo inicial de presets y firmware con valoraciones asociadas.

2.3.3 Auth y roles

La autenticación se gestiona mediante Laravel Sanctum en modo SPA, con cookies de sesión con estado. La instancia global de Axios está configurada para adjuntar credenciales automáticamente en cada petición. Los flujos de registro e inicio de sesión solicitan previamente la cookie CSRF antes de enviar las credenciales, y gestionan la respuesta hidratando el contexto global de autenticación o propagando los errores de validación a la interfaz.

El control de acceso se estructura en tres niveles operativos a partir de la columna booleana `is_admin` de la tabla de usuarios.

El modo local permite utilizar la herramienta sin sesión activa, deshabilitando todas las interacciones con la base de datos. Es el modo de operación por defecto cuando el usuario no está autenticado.

El usuario estándar, validado por el middleware de Sanctum, puede gestionar su repositorio privado de presets sin restricciones y participar en el sistema de valoraciones del repositorio público.

El administrador dispone de privilegios de escritura y borrado global sobre el repositorio público y sobre el catálogo de firmware. Este nivel está protegido en el backend mediante la combinación del middleware de Sanctum y un middleware de verificación de rol adicional, de forma que la validación no depende exclusivamente del estado del cliente.

En la capa de enrutamiento, las rutas públicas de descarga conviven con un grupo de endpoints protegidos por Sanctum para la gestión de perfil, repositorios y valoraciones, y un subgrupo adicional restringido a administradores para la subida de firmware y la administración del repositorio público.

2.3.4 Seguridad

La seguridad de la aplicación se ha abordado de forma transversal, cubriendo tanto la capa web como la comunicación con el dispositivo hardware. A continuación se describen las medidas implementadas y las consideraciones pendientes para el entorno de producción.

Protección CSRF y configuración CORS: La comunicación entre el frontend React y el backend Laravel se gestiona mediante Axios con Sanctum, que incluye protección CSRF integrada. La configuración de CORS en config/cors.php restringe los orígenes permitidos al dominio de la aplicación en producción, evitando peticiones cruzadas no autorizadas.

Hashing de contraseñas: Las contraseñas se almacenan utilizando bcrypt con un coste de factor 12, estándar recomendado por OWASP para hashing de credenciales. Laravel aplica este algoritmo por defecto a través de su sistema de autenticación.

Protección contra inyección SQL: Todas las consultas a base de datos se realizan a través del ORM Eloquent, que utiliza prepared statements de forma transparente, eliminando la superficie de ataque para inyección SQL en los endpoints de presets, firmware y autenticación.

Protección XSS: React escapa por defecto todo el contenido dinámico renderizado en JSX, previniendo la ejecución de scripts maliciosos inyectados. No se utiliza dangerouslySetInnerHTML en ningún punto de la aplicación.

Autenticación con Laravel Sanctum: La autenticación todavía se gestiona mediante cookies de API de Sanctum, tras la migración de Inertia a Axios, para garantizar la compatibilidad con el entorno de escritorio Tauri, se migrará a un sistema de tokens. El token se adjunta en cada petición mediante cabecera Authorization: Bearer.

Rate limiting: Se aplica limitación de tasa sobre los endpoints de login, registro y recuperación de contraseña mediante el middleware ThrottleRequests de Laravel. Esto mitiga ataques de fuerza bruta y de enumeración de cuentas. Las respuestas del endpoint de recuperación de contraseña son genéricas independientemente de si el email existe en el sistema.

Tiempo de sesión y expiración: La sesión expira tras 120 minutos de inactividad. La configuración SESSION_EXPIRE_ON_CLOSE garantiza además que la sesión se invalida al cerrar el navegador. La revocación del token se ejecuta en el servidor en el momento del logout, no únicamente en el cliente.

Logs de auditoría: Se registran los intentos de autenticación fallidos, incluyendo dirección IP, marca temporal y credencial utilizada. Estos registros permiten detectar patrones de ataque y constituyen una trazabilidad mínima recomendada en aplicaciones con acceso a datos de usuario.

Política de contraseñas: Se aplica una política de complejidad mínima mediante la clase Password de Laravel, que exige longitud mínima, combinación de mayúsculas, minúsculas y caracteres numéricos. En el entorno de producción se recomienda activar adicionalmente la validación ->uncompromised(), que contrasta la contraseña contra la base de datos de Have I Been Pwned mediante k-anonimato, sin transmitir la contraseña en claro.

Verificación de email: El sistema de verificación de email está integrado mediante Breeze.

Seguridad en el dispositivo ESP32 — credenciales WiFi: Las credenciales WiFi se transmiten desde la aplicación al dispositivo a través de la conexión USB Serial y se almacenan en la partición NVS del ESP32 aplicando ofuscación XOR. Es importante precisar que esta técnica constituye ofuscación, no cifrado criptográfico: protege contra la lectura directa de la memoria flash en condiciones normales, pero no frente a un atacante con acceso físico al dispositivo y capacidad de análisis de bajo nivel.

Integridad de presets — CRC32: Cada preset incluye un checksum CRC32 calculado sobre sus parámetros, que se verifica tanto en el dispositivo como en la aplicación. Esta medida garantiza la integridad accidental de los datos, detectando corrupciones en tránsito o en almacenamiento. CRC32 no es resistente a manipulación intencionada, al no ser un algoritmo criptográfico; sin embargo, dado que los presets provienen exclusivamente de clientes autenticados, el vector de ataque es limitado en el contexto actual.

Aspectos pendientes:

Doble factor de autenticación (2FA): No implementado en la versión actual.

Forzado de HTTPS en producción: El despliegue en producción debe forzar HTTPS de forma obligatoria. Sin cifrado en tránsito, los Bearer Tokens de Sanctum quedarán expuestos a ataques de interceptación.

2.4 Diseño y UX

2.4.1 La repercusión actual del Retrofuturismo



El retrofuturismo es la manifestación cíclica de una tensión permanente en la cultura visual: la nostalgia por el futuro que nunca llegó. Como concepto, precede con décadas a los movimientos que habitualmente se asocian a él. Su raíz se encuentra en la imaginación especulativa de las décadas de 1920 a 1960, cuando arquitectos, diseñadores industriales e ilustradores proyectaban un porvenir de ciudades flotantes, transporte atómico y tecnología al servicio de la utopía doméstica: el Atompunk. Lo que hoy se consume como estética retrofuturista es, en esencia, la mirada contemporánea sobre aquel futuro que no se cumplió.

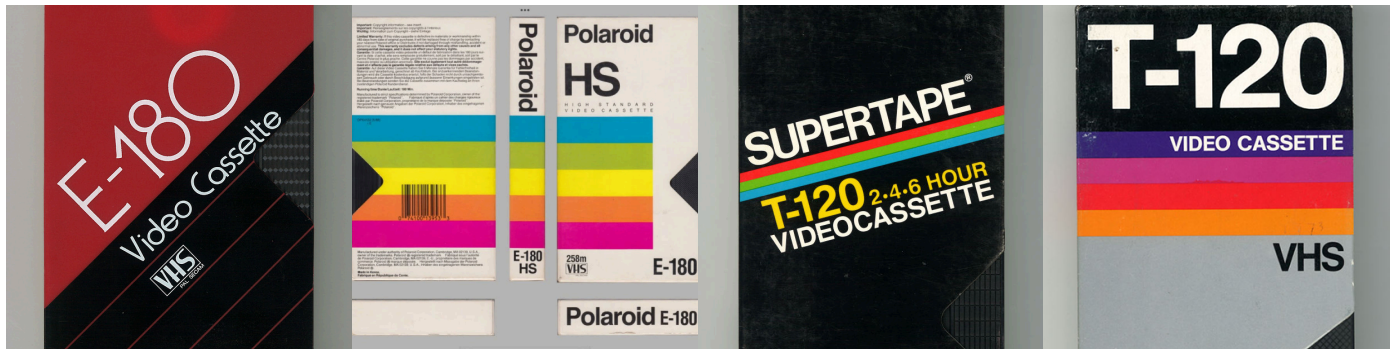
Su repercusión actual no puede entenderse como un fenómeno estético aislado. Es el resultado de una genealogía visual precisa, de unas condiciones culturales específicas y de la capacidad de internet para fragmentar, catalogar y redistribuir referencias visuales a una escala sin precedentes.



Para comprender el peso actual del retrofuturismo es imprescindible trazar su línea de origen. El Constructivismo ruso de los años veinte introdujo los principios que, con distintas transformaciones, atraviesan toda la tradición visual que hoy se recupera: geometría radical, asimetría, uso estructural del color plano y tipografía de función exclusivamente comunicativa, liberada de ornamento.

Estas ideas migraron hacia la Bauhaus alemana a través de figuras como László Moholy-Nagy y Wassily Kandinsky, donde se integraron en una filosofía de diseño orientada a la

producción industrial. El resultado fue el Estilo Internacional o Diseño Gráfico Suizo, que durante las décadas de 1950 y 1970 perfeccionó el sistema de retículas, la tipografía sans-serif y la sobriedad compositiva que hoy identificamos como modernismo clásico.



Este lenguaje formal llegó al consumo masivo de los años ochenta y noventa a través del diseño de producto de grandes fabricantes electrónicos como Sony, JVC, Philips o Braun. Las carátulas de cassettes y cintas VHS vírgenes, los manuales de usuario, el empaquetado de componentes: todo ese universo gráfico aplicó el funcionalismo heredado de la Bauhaus al objeto cotidiano, produciendo una estética que hoy resulta simultáneamente familiar y distante.

El retrofuturismo actual no es un movimiento unitario sino un conjunto de subgéneros con identidades visuales diferenciadas, cada uno de los cuales recupera una capa distinta de ese pasado tecnológico.



El Synthwave o Retrowave constituye la versión idealizada y cinematográfica de los ochenta: rejillas de perspectiva sobre horizontes violetas, tipografías cromadas, paletas de neón. Es una reconstrucción emocional de la época, más próxima a su representación cultural —el cine de ciencia ficción, la música electrónica— que a su realidad material.



La Blank Tape Aesthetic o Cassette Futurism se sitúa en el extremo opuesto. Recupera el objeto tecnológico cotidiano: la carcasa de plástico, los bloques de color funcional, las barras de identificación de formato. Su referencia no es la fantasía sino el producto. Es el diseño suizo aplicado al consumo masivo, y su revival actual responde a una nostalgia más concreta y menos espectacular que la del Synthwave.



El Vaporwave opera en una lógica diferente. Toma ese mismo material gráfico corporativo y lo somete a una distorsión deliberada mediante glitch digital, saturación cromática desplazada hacia el rosa y el turquesa, y yuxtaposición de elementos incongruentes. Si la Blank Tape Aesthetic recupera el objeto con fidelidad, el Vaporwave lo corrompe intencionadamente. Desde sus orígenes en comunidades de internet de principios de los años 2010, funciona simultáneamente como nostalgia y como crítica al consumismo que produjo esas imágenes.



El Steampunk y el Cyberpunk, aunque anteriores en su origen literario a buena parte de estas corrientes, se inscriben también en el paraguas retrofuturista. El primero proyecta un futuro alternativo desde la tecnología del siglo XIX; el segundo, nacido como distopía del presente en

los ochenta, es hoy consumido con la misma distancia nostálgica que el resto de estas estéticas. La estética Y2K y el revival del Frutiger Aero, por su parte, representa la capa más reciente de este proceso: la nostalgia por el cambio de milenio, con sus plásticos translúcidos, sus interfaces de Windows y su optimismo digital prematuro.

¿Por qué resurge ahora con tanta intensidad? Varias condiciones convergen para explicarlo.

La primera es estructural: internet ha creado los mecanismos para que las referencias visuales se archiven, circulen y sean apropiadas con una velocidad y una granularidad imposibles en cualquier época anterior. Plataformas como Pinterest, Tumblr o Instagram han funcionado como catalizadores de microcomunidades estéticas capaces de identificar, nombrar y desarrollar subgéneros con una precisión que antes requería décadas de crítica especializada. El Consumer Aesthetics Research Institute es un ejemplo de esta institucionalización espontánea del análisis de tendencias desde la comunidad.

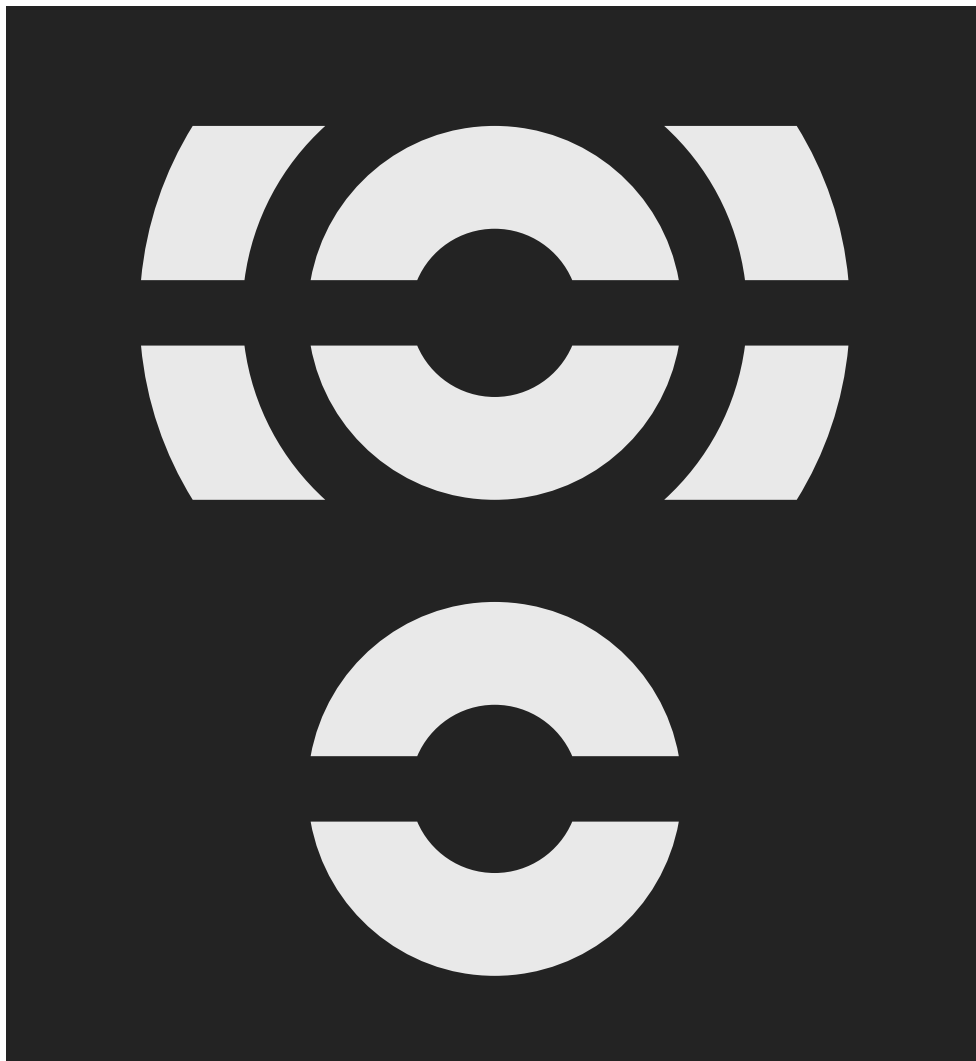
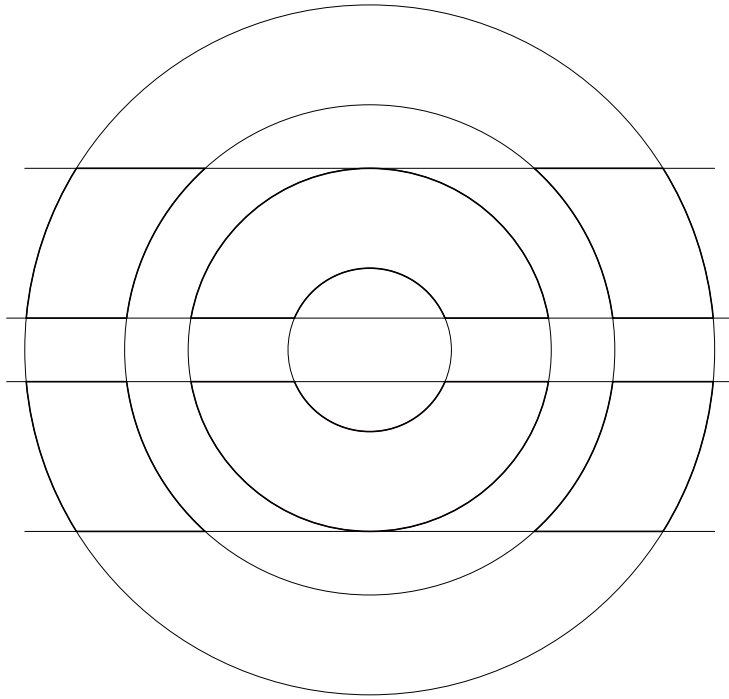
La segunda condición es cultural. El retrofuturismo prospera en momentos de incertidumbre sobre el futuro. La mirada hacia futuros pasados —hacia cómo se imaginaba el mañana desde un ayer ya conocido— ofrece una forma de relación con el tiempo históricamente específica: no es escapismo puro, sino una forma de procesar la aceleración tecnológica actual a través de la distancia que proporciona lo analógico.

La tercera es productiva. Las herramientas de diseño vectorial y la disponibilidad de tipografías históricas digitalizadas han reducido la barrera técnica para producir piezas con estos lenguajes visuales. Diseñadores independientes, músicos y desarrolladores de videojuegos han encontrado en estas estéticas un vocabulario accesible y culturalmente cargado.

La influencia del retrofuturismo en el diseño actual es transversal. En la identidad visual, marcas de muy distintos sectores han adoptado elementos de la Blank Tape Aesthetic o del Synthwave para comunicar valores como autenticidad, artesanía o independencia respecto a la homogeneización del diseño digital contemporáneo. En el diseño de interfaces, la recuperación de tipografías monoespaciadas, paletas de color reducidas y composiciones de retícula rígida aparece tanto en aplicaciones de productividad como en herramientas para desarrolladores, donde la referencia a las interfaces de terminal de los años ochenta funciona como señal de identidad técnica. En la música, el Synthwave ha consolidado un ecosistema de producción, portadas y videoclips con un lenguaje visual completamente codificado. En los videojuegos, el pixel art y la estética de pantalla CRT han pasado de ser una limitación técnica a ser una decisión estética consciente y valorada.

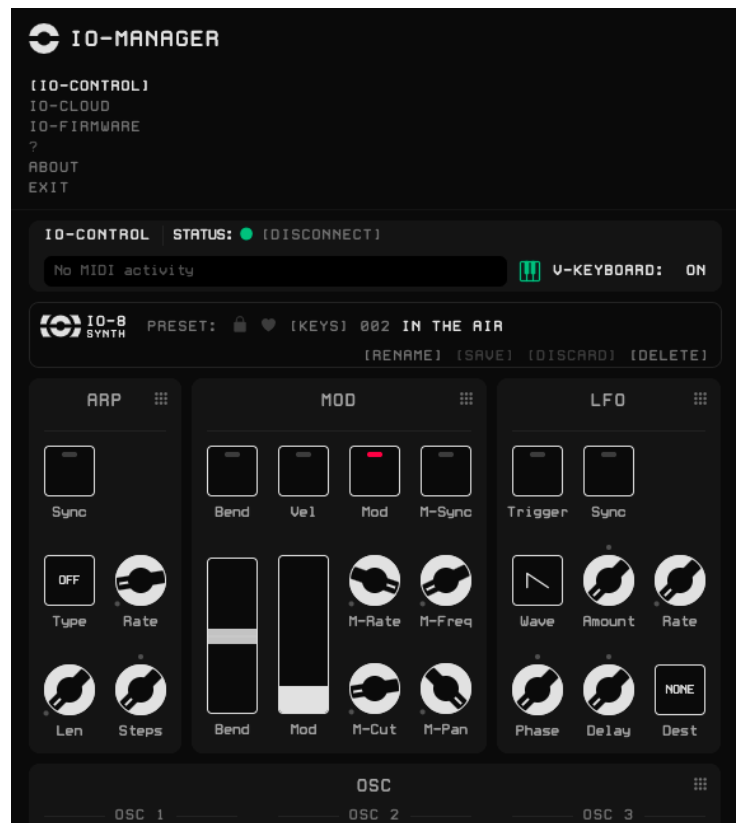
Lo que permanece constante en todas estas variantes es la misma tensión de origen: la distancia entre el futuro que se imaginó y el presente que se habita. Mientras esa distancia exista, el retrofuturismo seguirá siendo un lenguaje vigente.

2.4.2 Logo y línea de diseño



2.4.3 Adaptabilidad

La interfaz de la aplicación está construida sobre un sistema de layout flexible que se adapta de forma fluida a distintas resoluciones de pantalla. Los componentes principales utilizan flex y flex-wrap de Tailwind CSS, lo que permite que elementos como la barra de estado, la barra de presets o los controles de conexión se reorganicen automáticamente cuando el espacio disponible se reduce, sin necesidad de breakpoints explícitos en la mayoría de los casos. Esta decisión de diseño mantiene la interfaz funcional y legible tanto en monitores de escritorio amplios como en pantallas más reducidas, que es el entorno habitual de trabajo con hardware musical.



El panel de control incorpora una funcionalidad de reorganización dinámica de módulos implementada con la librería dnd-kit. Cada módulo del sintetizador —osciladores, LFO, envolventes, efectos y el resto de secciones— puede ser reposicionado por el usuario mediante arrastre, adaptando el layout a su flujo de trabajo personal. La implementación utiliza SortableContext con estrategia rectSortingStrategy, lo que calcula automáticamente la posición de destino en función de la geometría real de los módulos, sin asumir una cuadrícula fija. El manejador de arrastre está desacoplado del contenido del módulo mediante un botón independiente con touch-none, garantizando compatibilidad con dispositivos táctiles sin interferir con la interacción normal de los controles.

La aplicación está íntegramente redactada en inglés, una decisión justificada por el perfil del proyecto. IO Manager se dirige a músicos, makers y entusiastas de la síntesis digital, una comunidad de alcance internacional en la que el inglés es la lengua franca consolidada. La terminología técnica de síntesis sustractiva no tiene equivalentes en español de uso extendido, por lo que mantener la interfaz en inglés garantiza coherencia con el vocabulario que el usuario ya maneja en cualquier otro sintetizador o DAW. Igualmente, la traducción está contemplada como feature para el futuro.

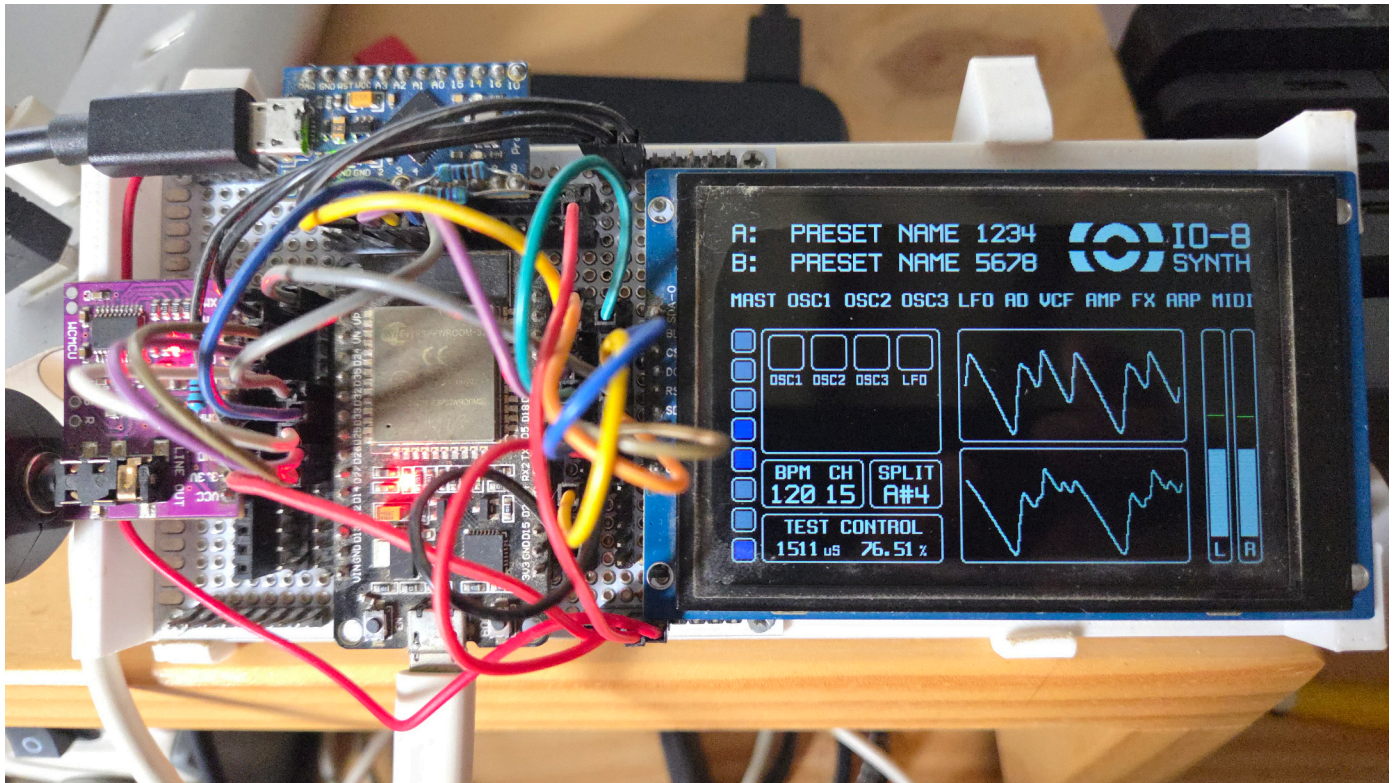
2.4.4 Accesibilidad

La accesibilidad se ha tratado como un requisito transversal y no como un añadido posterior. Toda la aplicación es navegable mediante teclado: los elementos interactivos siguen un orden de foco coherente con el flujo visual, y los paneles desplegados responden a eventos de teclado además de los de ratón. La barra de presets, por ejemplo, se cierra tanto al hacer clic fuera del contenedor como al pulsar Escape, gestionando ambos casos desde un único listener registrado en `useEffect`.

Los atributos ARIA están presentes de forma sistemática en los componentes que lo requieren. Las regiones funcionales están delimitadas con `role` y `aria-label`, los elementos decorativos se marcan con `aria-hidden="true"` para excluirlos de los lectores de pantalla, y los logs en tiempo real de actividad MIDI y estado de conexión utilizan `aria-live="polite"` para notificar cambios sin interrumpir el foco activo. Los botones de acción incluyen `aria-label` cuando su contenido visible no es suficientemente descriptivo, y los controles con estado binario utilizan `aria-pressed`.

El sistema de tooltips es una solución propia construida sobre el atributo nativo `title`, controlada mediante un estado global en `showTips.js`. El módulo expone un objeto `globalConfig` y una función `toggleGlobalTooltips` que alterna su valor y emite un evento personalizado `tooltips-changed` en `window`. Cada botón de la aplicación escucha ese evento mediante `addEventListener` en su propio `useEffect` y decide si adjunta o no el atributo `title` en función del estado actual. El resultado es un sistema de ayuda contextual que se activa y desactiva globalmente en tiempo real, sin necesidad de contexto de React ni prop drilling, y que aprovecha el comportamiento nativo del navegador para mostrar los textos de ayuda.

2.4 Integración ESP32



Aunque lo que respecta a la programación de la ESP32 no es concerniente para este proyecto, sí que he considerado dar unos apuntes sobre el código que escribí para vincular la placa con la App.

Servidor WebSocket

La comunicación en tiempo real entre la aplicación y el dispositivo se apoya en un servidor WebSocket implementado desde cero sobre `esp_http_server`, la pila HTTP nativa del ESP-IDF. La decisión de no utilizar librerías de terceros como ESPAsyncWebServer responde a limitaciones de estabilidad y compatibilidad con el scheduler de FreeRTOS bajo carga de audio en tiempo real.

El servidor gestiona hasta cuatro clientes concurrentes mediante una tabla de sesiones de tamaño fijo. Cada sesión se modela como una estructura que registra el descriptor de fichero del socket, un estado de ciclo de vida (`IDLE`, `CONNECTED`, `CLOSING`) y los contadores de keepalive: marca temporal del último ping enviado, del último pong recibido y número de pongs perdidos consecutivos. El acceso a esta tabla está protegido por un mutex de FreeRTOS para prevenir condiciones de carrera entre el hilo del servidor HTTP y las tareas de audio que corren en los dos núcleos del ESP32.

El handler principal gestiona todos los tipos de frame definidos por RFC 6455: texto, binario, ping, pong y close. Los frames de ping reciben respuesta inmediata con el mismo payload. Los pong actualizan los contadores de keepalive de la sesión correspondiente. Los frames de cierre ejecutan la secuencia de terminación limpia: notificación al callback de aplicación, envío del frame

de close al cliente, cierre del socket mediante ``httpd_sess_trigger_close`` y liberación de la entrada en la tabla de sesiones.

El módulo expone tres callbacks configurables en tiempo de registro: ``on_post_handshake``, invocado justo después del upgrade HTTP a WebSocket; ``on_data``, que recibe cada frame de datos junto con la sesión de origen; y ``on_close``, notificado ante cualquier cierre de conexión. Esta separación entre el transporte y la lógica de aplicación permite mantener ``ws_server.h`` como una capa reutilizable independiente del protocolo de presets.

Protocolo de aplicación

Sobre el servidor WebSocket se define un protocolo binario de paquetes de tamaño fijo de 32 bytes: un byte de opcode seguido de 31 bytes de payload. Los opcodes activos son ``MSG_SAVE`` (0xFA), ``MSG_LOAD`` (0xFB), ``MSG_SEND_DATA`` (0xFC), ``MSG_UPLOAD`` (0xFD), ``MSG_DELETE`` (0xFE) y ``MSG_HEARTBEAT`` (0xFF).

Inmediatamente después del handshake, el dispositivo transmite de forma proactiva el estado completo al cliente: primero los 2.688 bytes de metadatos de los 128 slots en chunks de 31 bytes, y a continuación los 128 bytes del preset activo con el mismo mecanismo. Este diseño elimina cualquier petición de sincronización inicial desde la aplicación y garantiza que el cliente dispone del estado completo del dispositivo antes de procesar ninguna interacción del usuario. Los mensajes MIDI entrantes se procesan mediante un buffer circular de tamaño potencia de dos, lo que permite avanzar el índice de escritura con una operación AND en lugar de una división modular, relevante en el contexto de tiempo real. La validación de cada mensaje verifica que el byte de estado esté en el rango 0x80–0xEF antes de encolarlo.

Sistema de ficheros y gestor de presets

El almacenamiento de presets utiliza LittleFS sobre la partición de datos de la flash del ESP32. Cada preset ocupa un fichero binario de 128 bytes con una estructura fija definida por offsets constantes: cabecera de dos bytes (10'), identificador de slot, byte de flags empaquetado, nombre de 16 bytes, 104 bytes de parámetros y CRC32 de cuatro bytes calculado exclusivamente sobre los parámetros.

El byte de flags codifica cuatro valores booleanos en los bits bajos — ``Empty``, ``ReadOnly``, ``Fav``, ``Exists`` — y la categoría del preset en los tres bits altos, lo que permite almacenar ocho categorías sin coste adicional de espacio. El acceso a flags individuales se implementa mediante operaciones de máscara, con una distinción explícita entre los flags booleanos y el campo de categoría de tres bits.

El CRC32 se calcula únicamente sobre el bloque de parámetros, no sobre la cabecera ni los metadatos. Esto permite verificar la integridad del contenido sonoro de forma independiente del estado administrativo del slot, y es el mismo valor que se utiliza como identificador de contenido en la base de datos de la aplicación para detectar duplicados.

Para evitar recorrer los 128 ficheros en cada arranque, el sistema mantiene un fichero de metadatos (/metadata) que contiene una caché serializada con los flags, el nombre y el CRC32 de cada slot. Esta estructura ocupa 21 bytes por slot, 2.688 bytes en total, y se carga íntegramente en RAM al arrancar. Las operaciones de escritura de metadatos individuales utilizan un offset calculado sobre el identificador del slot para actualizar únicamente los 21 bytes correspondientes sin reescribir el fichero completo.

El identificador del slot activo se persiste en la partición NVS, separada del sistema de ficheros de presets, lo que permite recuperar el estado anterior al reinicio de forma independiente. La secuencia de arranque comprueba la integridad del sistema mediante `sysCheck()`, que valida la existencia del fichero de metadatos y que todos los slots tienen el flag `Exists` activo. Si la comprobación falla, se ejecuta una inicialización completa que formatea la partición y crea los 128 ficheros vacíos con su estructura correcta.

3. CONCLUSIONES

3.1 Análisis de resultados

El proyecto ha alcanzado sus objetivos principales en su totalidad. IO Manager es una aplicación funcional, documentada y compilable como binario de escritorio que cubre el ciclo completo de uso de un instrumento IO: primera conexión por USB, carga de firmware, provisión de credenciales WiFi, control en tiempo real por WebSocket y gestión de presets en tres capas —dispositivo, repositorio privado y repositorio público—.

El entorno de desarrollo se automatizó desde las primeras sesiones mediante scripts de Automator para el arranque y cierre del stack Docker/Sail, y se configuró un acceso remoto completo por SSH, VNC y SMB que permitió trabajar sobre el mismo entorno desde dos máquinas distintas. Este nivel de automatización contribuyó a mantener la consistencia del entorno a lo largo de las cien sesiones de desarrollo.

En el lado del backend, Laravel con Sanctum ofrece una API RESTful con autenticación por token, control de acceso por roles, rate limiting, verificación de email y logs de auditoría. La base de datos cubre las cuatro entidades necesarias —usuarios, presets, valoraciones y firmware— con relaciones bien definidas y borrado en cascada. El sistema de seeders permite inicializar el entorno de desarrollo de forma determinista en un único comando.

La integración con el hardware ha sido la parte técnicamente más exigente del proyecto. El servidor WebSocket implementado desde cero sobre ``esp_http_server`` del ESP-IDF gestiona sesiones concurrentes con control de keepalive, un protocolo binario de paquetes fijos y sincronización de estado por mutex de FreeRTOS. El sistema de ficheros sobre LittleFS almacena 128 presets de 128 bytes con una caché de metadatos en RAM que evita recorrer el sistema de ficheros en cada arranque. La latencia de control en tiempo real se consiguió desactivando el modem sleep del chip WiFi, logrando una respuesta suficiente para uso musical interactivo.

El provisioning WiFi a través de la consola serie sigue un protocolo de máquina de estados con timeout, ofuscación XOR de la contraseña en tránsito y almacenamiento en la partición NVS. El flasheo de firmware desde la aplicación, que descarga el binario directamente a memoria sin pasar por disco y lo transfiere mediante esptool-js a través de la Web Serial API, cierra el ciclo de actualización del dispositivo sin herramientas externas.

La interfaz ha recibido un tratamiento completo de accesibilidad con atributos ARIA, navegación por teclado, sistema de tooltips global activable en tiempo real, y estilos Tailwind con composiciones flex que mantienen la funcionalidad en distintas resoluciones.

La documentación cubre el código PHP mediante Scribe, el código JavaScript mediante JSDoc con tema docdash, diagramas de dependencias generados con dependency-cruiser, el esquema de base de datos exportado y el presente cuaderno de bitácora con registro de las cien sesiones de trabajo.

3.2 To do

Los elementos pendientes identificados al cierre del proyecto son los siguientes, ordenados por prioridad técnica.

HTTPS en producción: Es el único pendiente con carácter bloqueante para cualquier despliegue real. Sin cifrado en tránsito, los tokens de Sanctum viajan en claro. Requiere configurar HSTS en el servidor web y activar el forzado de esquema en el ``AppServiceProvider`` de Laravel.

Rama serial en Tauri: La integración de ``tauri-plugin-serialplugin`` está iniciada y el plugin añadido al proyecto, pero la reimplementación de la lógica de conexión serial bajo el contexto de Tauri está pendiente. En la build de escritorio, la Web Serial API del navegador queda fuera del alcance de Tauri, por lo que esta funcionalidad requiere reimplementarse usando las APIs nativas del plugin de Rust.

Traducción: La interfaz está preparada para internacionalización al estar los textos en inglés de forma consistente, pero no existe actualmente ningún sistema de i18n implementado. Añadir soporte para español requeriría adoptar una librería como ``react-i18next`` y externalizar las cadenas de texto.

Adaptación a tablet: El layout actual está optimizado para escritorio. Los componentes de control —knobs, sliders, botones— necesitarían revisión de tamaños de área táctil y reorganización del grid de módulos para resoluciones intermedias.

Manual de usuario: La documentación técnica del código está completa, pero no existe documentación orientada al usuario final que describa el flujo de uso desde la primera conexión hasta la gestión del repositorio en la nube.

2FA: La implementación de doble factor de autenticación mediante TOTP es una mejora de seguridad identificada pero no prioritaria para el alcance actual del proyecto. La librería ``pragmarx/google2fa-laravel`` ofrece integración directa con Laravel.

Administrador de usuarios: El panel de administración cubre el repositorio público y el catálogo de firmware, pero no expone herramientas de gestión de cuentas de usuario —listado, suspensión, borrado— desde la interfaz. Esta funcionalidad está disponible en el backend mediante Eloquent pero no tiene frontend asociado.

4. DOCUMENTACIÓN

ANEXOS

WEBGRAFÍA

PROYECTO, MEMORIA, PRESENTACIÓN:

Proyecto en Github

<https://github.com/gus-fernandez/io-manager>

Memoria del Proyecto

<https://github.com/gus-fernandez/io-manager/blob/main/public/IO-Manager-Memo.pdf>

Presentación

<https://docs.google.com/presentation/d/1LsovoQkOHvCWJamh4apjGqDHOqYsFfXw31W0cmszkZo>

DOCUMENTACIÓN:

Documentación PHP

<https://htmlpreview.github.io/?https://github.com/gus-fernandez/io-manager/blob/main/public/docs/index.html>

Documentación JS

<https://htmlpreview.github.io/?https://github.com/gus-fernandez/io-manager/blob/main/public/docs-js/index.html>

Cuaderno de Bitácora

<https://github.com/gus-fernandez/io-manager/blob/main/public/docs-misc/Cuaderno-bit.pdf>

ANEXOS:

Gráfico routes Laravel

<https://github.com/gus-fernandez/io-manager/blob/main/public/docs-misc/Routes-middleware.pdf>

Schema SQL

<https://github.com/gus-fernandez/io-manager/blob/main/public/docs-misc/schema.sql>

Carpeta Graphs React

<https://github.com/gus-fernandez/io-manager/tree/main/public/docs-misc/graphs>

Paths React

<https://github.com/gus-fernandez/io-manager/blob/main/resources/js/Features/Device/Shared/utils/paths.txt>

Reporte de métricas React

<https://github.com/gus-fernandez/io-manager/blob/main/public/docs-misc/metrics.txt>

Drafts Sintetizador

<https://github.com/gus-fernandez/io-manager/tree/main/public/docs-misc/drafts>

WEBGRAFÍA:

Marcel Licence

<https://github.com/marcel-licence>

<https://www.youtube.com/@marcellicence/videos>

<https://www.matrixsynth.com/search/label/Marcel%20Licence>

Dexed

<https://www.futuremusic-es.com/dexed-legendario-yamaha-dx7-se-reencarno-sofsinte-ademas-gratuito/>

https://github.com/probonopd/Synth_Dexed

<https://github.com/asb2m10/dexed>

<https://www.hackster.io/news/simon-peter-s-mini-dexed-cramps-eight-simulated-yahama-dx7-synths-into-a-single-raspberry-pi-3c656b2bf01c>

<https://synthanatomy.com/2025/10/dtronics-dt-dx-hardware-version-of-the-free-dexed-fm-synthesizer-plugin.html>

Audacity

https://en.wikipedia.org/wiki/Audacity_%28audio_editor%29

<https://arstechnica.com/gadgets/2021/07/no-open-source-audacity-audio-editor-is-not-spyware>

<https://www.techradar.com/news/audacity-reverses-opt-in-telemetry-plans-following-user-revolt>

<https://www.gadgets360.com/apps/news/audacity-privacy-policy-update-spyware-clarification-owner-muse-group-2480371>

https://www.cei-bg.org/sites/default/files/National_report_on_good_practices_Bulgaria_%5BEN%5D.pdf

<https://softlibre.unizar.es/manuales/aplicaciones/audacity/audacity-manual-1.2.pdf>

<https://arxiv.org/abs/2110.13323>

Desarrollo

<https://laravel.com/docs>

<https://laravel.com/docs/sail>

<https://laravel.com/docs/starter-kits#laravel-breeze>

<https://laravel.com/docs/sanctum>

<https://es.react.dev/>

<https://es.react.dev/learn/managing-state>

<https://es.react.dev/learn/adding-interactivity>

<https://vite.dev/>

<https://tailwindcss.com/docs>

<https://tauri.app/>

<https://www.npmjs.com/package/tauri-plugin-serialplugin-api>

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/esp_http_server.html

<https://www.npmjs.com/package/esptool-js>

<https://github.com/ESP32Async/ESPAsyncWebServer>

<https://esp32async.github.io/ESPAsyncWebServer/>

<https://datatracker.ietf.org/doc/html/rfc6455>

https://en.wikipedia.org/wiki/Nagle%27s_algorithm

<https://dndkit.com/>

<https://axios-http.com/docs/intro>

<https://scribe.knuckles.wtf/laravel>

<https://jsdoc.app/>

<https://snyk.io/es/blog/tanstack-npm-packages-compromised/>

<https://www.svgrepo.com/>

<https://favicon.io/favicon-converter/>

Diseño

<https://es.wikipedia.org/wiki/Suprematismo>

<https://medium.com/@bhaktirathod90/how-retro-futurism-shapes-modern-design-d9a65366cd98>

<https://es.pinterest.com/ideas/minimal-techno/939183652201/>

<https://tvtropes.org/pmwiki/pmwiki.php/Main/CassetteFuturism>

https://aesthetics.fandom.com/wiki/Cassette_Futurism

<https://eyeondesign.aiga.org/from-ignored-ubiquity-to-design-classic-the-art-of-blank-vhs-tape/>

https://en.wikipedia.org/wiki/Minimal_techno

<https://zarmatype.com/the-power-of-fonts-in-retro-futurism-combining-vintage-aesthetics-with-modern-design/>

<https://www.artsy.net/article/artsy-editorial-constructivism-brought-russian-revolution-art>

<https://www.dximagazine.com/2018/06/08/ernst-keller-pionero-del-estilo-suizo/>

<https://www.laimprentacg.com/ernst-keller-estilo-suizo/>

<https://99designs.com/blog/design-history-movements/memphis-design/>

<https://hommes.studio/journal/what-is-memphis-design-style/>

<https://www.aesdes.org/2024/01/24/atompunk/>

<http://www.cobaltjade.com/2021/03/atompunk/>

<https://aesthetics.fandom.com/wiki/Atompunk>

https://aesthetics.fandom.com/wiki/Raygun_Gothic

<https://www.hispasonic.com/productos/roland-jupiter-8/20026>

